DEBRECENI EGYETEM
INFORMATIKAI KAR

SZAKDOLGOZAT

Párhuzamos képfeldolgozó rendszer fejlesztése és teljesítményvizsgálata Rust nyelven

Témavezető:	Készítette:
Dr. Tornai Róbert	Farkas Beatrix
Egyetemi docens	Gazdaságinformatikus Bsc

DEBRECEN

2026

TARTALOMJEGYZÉK

1. Bevezetés 2

2. Elméleti háttér és technológiai alapok 4
   - 2.1. Kutatómunka: A processzorok korlátai és az órajelfal 5
   - 2.2. Párhuzamosítás CPU-n: a többszálúság és a tipikus problémák 6
   - 2.3. A digitális képfeldolgozás alapjai 7
   - 2.4. A videókártyák (GPU) ereje és a Compute Shaderek 8

3. Gyakorlati megvalósítás és teljesítményelemzés 10
   - 3.1. A felhasznált technológiák és a fejlesztői környezet 11
   - 3.2. A szoftver architektúrája és a memóriakezelés 12
   - 3.3. CPU alapú párhuzamosítás a Rayon segítségével 14
   - 3.4. GPU alapú gyorsítás Compute Shaderekkel 15
     - 3.4.1. Komplex térbeli szűrők GPU-s megvalósítása: A Sobel-operátor 18
   - 3.5. Robusztus hibakezelés és memóriabiztonság a gyakorlatban 20

4. Teljesítménymérések és eredmények 21
   - 4.1. Vizuális eredmények és algoritmikus transzformációk 23
   - 4.2. Futtatási környezet és parancssori paraméterezés 27
   - 4.3. Hardveres terhelésvizsgálat és a végrehajtási idő elemzése 28
   - 4.4. Képfeldolgozási mérési jelentés és teljesítmény-elemzés 29
   - 4.5. A szoftver korlátai és jövőbeli továbbfejlesztési lehetőségek 32

5. Összefoglalás 33

6. Irodalomjegyzék 34

7. Ábrajegyzék 36

1. Bevezetés

Amikor elérkezett a pillanat, hogy szakdolgozati témát kell választanom, egy dologban teljesen biztos voltam, hogy mindenképpen valami informatikai, és ne gazdasági tematikájú feladatot válasszak. Gazdaságinformatikus hallgatóként az egyetemi éveim alatt betekinthettem mindkét világba, de egy kis idővel egyre világosabbá vált számomra, hogy a programozás és a gyakorlati feladatok sokkal jobban érdekelnek, mint az elméleti vagy gazdasági tárgyak. Úgy éreztem, ez a szakdolgozat a tökéletes alkalom arra, hogy egy olyan témát válasszak, ami kizökkent a komfortzónámból, és igazi mérnöki kihívást jelent. Kezdetben viszont eléggé elveszve éreztem magam a rengeteg lehetőség között, és nem volt egy konkrét, kiforrott ötletem arra, hogy mi is legyen az pontosan. Miután kiválasztottam ezt a témát, segítséget kértem a témavezetőmtől, aki hatalmas támaszt jelentett. Átbeszéltük a részleteket, ő pedig segített pontosítani a feladatokat és kijelölni a konkrét irányt. Ez a beszélgetés adta meg azt a kezdőlökést, amire szükségem volt, kijelölt egy tiszta utat, amin el tudtam indulni, és onnantól kezdve már alig vártam, hogy ténylegesen elkezdhessek dolgozni a projekten. Ahogy elkezdtem belemélyülni a munkába, hamar kiderült, hogy a témavezetőm nem túlzott, ez tényleg óriási kihívás. A Rust nyelv szigorú logikája, a memóriakezelés megértése, és az, hogy hogyan lehet egy képet szálakra bontva, a processzoron (CPU) és a videókártyán (GPU) egyszerre futtatni, az elején kifejezetten nehéznek tűnt. Viszont ahogy teltek a hetek, és sikerült megírnom az első saját, párhuzamosított szűrőimet például egy egyszerű negatív effektet vagy egy szürkeárnyalatosítást. Nagyon jó érzés volt látni a gyors előrehaladást, ami még több motivációt adott a folytatáshoz. Menet közben jöttem rá, hogy a mai gépek hiába nagyon erősek, a programok többsége mégis lassan fut, mert nem használja ki az összes magot. A processzorok órajele a fizikai korlátok miatt már nem igazán növelhető tovább, ha egy szoftvert különösen egy hatalmas adatmennyiséggel dolgozó képfeldolgozót gyorsítani akarunk, akkor a többmagos architektúrákat kell okosan kihasználnunk. A hagyományos programozási nyelvekkel a párhuzamosítás gyakran instabilitáshoz vagy nehezen megtalálható memóriahibákhoz vezet. A Rust viszont pont az ellenkezője mivel, ezt a folyamatot biztonságosan, mégis hihetetlenül gyorsan képes végrehajtani. Ez volt az a pont, ahol a személyes kíváncsiságom sokkal jobban felerősödött. A szakdolgozatom célja tehát az lett, hogy saját kezűleg tapasztaljam meg és mérjem le ezt a technológiát. Létrehoztam egy parancssorból vezérelhető (CLI) alkalmazást, amely többféle klasszikus képtranszformációt mint például a fényerőnövelés, a Sobel-féle éldetektálás vagy a Box Blur homályosítás képes lefuttatni. Nemcsak a kód megírása volt a célom, hanem az is, hogy pontosan összehasonlítsam a sebességüket. Látni akartam, hogy mennyivel gyorsabb egy hagyományos, egyszálú megoldásnál az, ha a CPU minden magját kihasználom, vagy ha egyenesen a videókártya több ezer magjára küldöm a számításokat. Ezt a programom futás közben ezredmásodperces pontossággal méri, és egy külön fájlba menti az eredményeket. A projekt gyakorlati megvalósításához külön motivációt jelentett, hogy a fejlesztést és a méréseket egy modern, kifejezetten erős gépen (egy 8 magos AMD Ryzen 7 9800X3D processzorral és egy 16 GB VRAM-mal rendelkező NVIDIA RTX 5070 Ti videókártyával) végezhettem el. Egy átlagos hardveren a párhuzamosításból származó sebességkülönbség kevésbé lett volna látványos. Ezen a rendszeren viszont már saját szememmel, ezredmásodperces pontossággal tapasztalhattam meg, hogy mekkora elképesztő erőt jelent, ha egy algoritmust nem csupán egyetlen szálon, hanem a CPU összes magját, vagy épp a GPU több ezer számolóegységét kihasználva futtatok le. A dolgozatomat pontosan úgy építettem fel, hogy lépésről lépésre bemutassa ezt az egész folyamatot. A második fejezetben az elméleti hátteret a többmagos architektúrák működését, a digitális képfeldolgozás alapjait, valamint a Rust nyelv sajátosságait tekintem át. A harmadik fejezet a gyakorlati megvalósításra fókuszál, itt mutatom be a saját fejlesztésű alkalmazásom felépítését, a felhasznált algoritmusokat és a szoftverfejlesztés során felmerült technológiai kihívásokat. Végül a negyedik fejezetben a különböző hardveres megközelítések (CPU és GPU) teljesítményét hasonlítom össze a konkrét mérési eredmények kiértékelésével.

2. Elméleti háttér és technológiai alapok

Mielőtt belevágtam volna a képfeldolgozásba és a feladatok párhuzamosításába, először át kellett tekintenem, hogyan is működnek a mai számítógépek, és milyen fizikai korlátaik vannak. Ezekből az alapokból gyorsan kiderült, hogy miért pont a Rust nyelv és a videókártya a legjobb választás erre a feladatra. Ebben a fejezetben azokat az elméleti dolgokat foglalom össze, amiket mindenképpen meg kellett értenem a saját programom megírásához.

2.1. Kutatómunka: A processzorok korlátai és az órajelfal

Régebben a programozóknak még nem kellett annyira aggódniuk a sebesség miatt. Ha egy program lassan futott le, a fejlesztőknek sokszor elég volt várniuk egy-két évet, a hardvergyártók ugyanis évről évre gyorsabb processzorokat dobtak piacra. Ahogy utána olvastam a processzorok történelmének, folyamatosan szembejött velem a Moore-törvény és a Dennard-féle skálázódás. Ezek röviden azt jelentették, hogy a gyártók régen úgy tudták évről évre drasztikusan növelni a chipek sebességét, azaz az órajelet, hogy közben a processzorok áramfogyasztása nem nőtt meg annyira, hogy túlmelegedjenek.[1] Így a régebben megírt, hagyományos, egyszálú kódok anélkül futottak az újabb gépeken egyre gyorsabban, hogy magának a fejlesztőnek egyetlen sort is át kellett volna írnia a programjában. A kutatásom során azonban kiderült, hogy a kétezres évek közepére ez az úgynevezett kényelem megszűnt, mivel a gyártók beleütköztek egy durva fizikai határba, ami az órajelfal (clock speed wall) volt. Amikor megpróbálták a chipeket 3–4 GHz fölé gyorsítani, a processzorok hirtelen nagyon durván elkezdtek melegedni, és rengeteg áramot fogyasztottak. A probléma az volt, hogy egy szint fölött a chipek már annyira felmelegedtek, hogy a hagyományos ventilátoros hűtés nem volt elég, és a túlságosan apró alkatrészek miatt az áram is elkezdett szivárogni.[2] Ezután értettem meg azt, hogy egyetlen mag sebességét nem lehet a végtelenségig növelni, mert a processzor a végén szimplán elolvadna a hőtől. Mivel az órajel növelése már korlátozott volt, kénytelenek voltak más módszert alkalmazni, ami viszont az volt, hogy egyetlen szupergyors mag helyett inkább több önálló magot tettek egymás mellé a processzorban.[3] Megjelentek a kétmagos, négymagos, majd a mai, akár 8–16 vagy még több maggal dolgozó asztali processzorok. Ez a változás felforgatta a programozás világát. A gond az volt, hogy egy normál program csak egy szálon fut, így a sok magból mindig csak az egyiket fogja leterhelni. Tehát ha egy mai, 8 magos gépen futtatunk egy ilyen hagyományos kódot, a processzor erejének alig 12%-át használjuk, a többi mag pedig lényegében kihasználatlan marad. A gyorsítás egyetlen útja a feladatok darabolása és a párhuzamosítása. Vagyis ahhoz, hogy egy program tényleg gyors legyen, már a kódolásnál oda kell figyelni, hogy a szoftver úgymond szétossza a munkát a rendelkezésre álló magok között.

---

[1] Mark Bohr (2007): A 30 Year Retrospective on Dennard's Scaling Theory. IEEE Solid-State Circuits Society Newsletter. - https://www.eng.auburn.edu/~agrawvd/COURSE/READING/LOWP/Boh07.pdf

[2] Herb Sutter (2005): The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. Dr. Dobb's Journal. - http://www.gotw.ca/publications/concurrency-ddj.htm

2.2. Párhuzamosítás CPU-n: a többszálúság és a tipikus problémák

Ahhoz, hogy egy program ténylegesen ki tudja használni a mai többmagos processzorokat, a feladatokat úgynevezett szálakra (threads) kell bontani. Amikor elindítunk egy sima, hagyományos programot, az operációs rendszer ad neki egyetlen fő szálat. Hogy egy egyszerű példát mondjak, ez pont olyan, mint egy konyha, ahol csak egyetlen szakács dolgozik. Hiába profi és gyors, a rendeléseket csak szépen egymás után tudja megcsinálni. A többszálú programozás (multithreading) pont arról szól, hogy felveszünk mellé még több “szakácsot”. Az én tesztgépemben például egy 8 magos Ryzen processzor van, így ott nyolc “szakács” tud egyszerre, párhuzamosan dolgozni a feladatokon. Elméletben persze mindez logikusan és egyszerűen hangzik, a gyakorlatban viszont a feladatok párhuzamosítása komoly odafigyelést és teljesen más gondolkodásmódot igényel. A szálak ugyanis ugyanazt a közös memóriát használják. Ha visszatérek a konyhás példára akkor a nyolc szakács egyetlen pulton, ugyanazokkal az alapanyagokkal és késekkel dolgozik. Ha nem korlátozzuk le hajszálpontosan, hogy ki, mikor és mihez nyúlhat, abból hatalmas káosz keletkezhet. A régebbi, hagyományos programozási nyelvekben (mint például a C vagy a C++) a többmagos programozás során két olyan gyakori hibát emelnék ki, amelyek a legtöbb gondot okozzák. Az egyik ilyen az ismert adatütközés vagy data race. Ez a hiba akkor jön elő, amikor két szál a másodperc töredéke alatt egyszerre akarja módosítani pont ugyanazt az adatot, úgy, hogy nem is tudnak egymásról. A képfeldolgozásnál ez olyan, mintha két szál egyszerre próbálná meg átírni ugyanannak a pixelnek a színét. Ilyenkor a végeredmény teljesen kiszámíthatatlan vagy egy hibás, zajos képet kapunk, vagy a program egyszerűen összeomlik a memóriahiba miatt. A másik tipikus hiba, a holtpont, azaz deadlock. Az adatütközések elkerülésére a szoftverfejlesztők úgynevezett zárolásokat (mutexeket) alkalmaznak. Ez azt jelenti, hogy egy szál lefoglalja magának az adott memóriarészt, amíg dolgozik vele, a többiek pedig addig várnak a sorukra. Viszont, ha a kódolásnál hiba csúszik ebbe a logikába, könnyen előfordulhat, hogy az “A” szál vár egy adatra, amit épp a “B” szál foglal, miközben a “B” szál egy olyan adatra vár, amit meg az “A” szál tart lefoglalva. Ilyenkor a két szál a végtelenségig vár egymásra, a program pedig teljesen lefagy.[4] A legrosszabb ezekben a hibákban az, hogy sokszor nem is jönnek elő minden futtatáskor. Teljesen véletlenszerűen, csak bizonyos terhelésnél vagy időzítésnél bukkannak fel, így a megkeresésük és javításuk rendkívül nehéz. Egy képfeldolgozó szoftvernél, ami több millió pixellel dolgozik egyszerre, ez az instabilitás megengedhetetlen. Pontosan emiatt a Rust nyelv egy teljesen más megközelítést alkalmaz, amivel már fordítási időben, a kód megírásakor kizárja ezeket a problémákat.[5]

---

[3] David A. Patterson - John L. Hennessy (2013): *Computer Organization and Design: The Hardware/Software Interface* (5. kiadás). Morgan Kaufmann.

2.3. A digitális képfeldolgozás alapjai

Mielőtt rátérnék arra, hogy hogyan lehet felgyorsítani a képfeldolgozást, tisztázni kell, hogy mi is az a kép a számítógép számára. Amikor mi a képernyőn ránézünk egy fotóra, formákat és színeket látunk, a processzor viszont csak egy hatalmas számtáblázatot (mátrixot) érzékel. A digitális képek apró, négyzet alakú pontokból, úgynevezett pixelekből állnak. Minden egyes pixelnek van egy konkrét színe, amit a legtöbb esetben az RGB (Red, Green, Blue = piros, zöld, kék) színmodell határoz meg. Ez a gyakorlatban azt jelenti, hogy egyetlen pixel színét három számérték írja le, amelyek 0 és 255 között mozoghatnak. Ha például egy pixel értéke (255, 0, 0), akkor az a pont teljesen piros, ha (255, 255, 255), akkor fehér, ha pedig (0, 0, 0), akkor fekete. Sokszor ehhez még hozzájön egy negyedik érték is, az alfa-csatorna (Alpha), ami azt mondja meg, hogy mennyire átlátszó az adott pont. Ezt hívjuk RGBA formátumnak.[6] A programozás szempontjából az az igazán érdekes, hogy a memóriában a kép nem egy kétdimenziós rácsként tárolódik, hiába látszik annak. A memóriában (RAM) a pixelek adatai egyetlen végtelenül hosszú, egydimenziós tömbként (array) helyezkednek el egymás után. Tehát egy 1920x1080 felbontású (Full HD) kép esetében nagyjából 2 millió pixel található, ami az RGBA értékek miatt egy több mint 8 millió számból álló, egybefüggő adatsort jelent. Amikor a programommal egy képtranszformációt (például egy negatív effektet vagy egy szürkeárnyalatosítást) csinálok, akkor valójában semmi mást nem teszek, mint hogy egy ciklus segítségével végigmegyek ezen a több milliós tömbön. Kiolvasom a számokat, elvégzek rajtuk egy matematikai műveletet, majd a kapott új értékeket visszamentem a memóriába. Ez a folyamat egyetlen magon futtatva iszonyatosan sokáig tarthat, főleg, ha mondjuk 4K vagy 8K felbontású képekkel dolgozunk. Viszont a képfeldolgozásnak van egy hatalmas előnye, az egyes pixelek színeinek kiszámítása a legtöbb esetben teljesen független egymástól. A gépnek nem kell tudnia, hogy milyen színű a kép bal felső sarka ahhoz, hogy kiszámolja a jobb alsó sarokban lévő pixel új színét. Ez a függetlenség teszi a képfeldolgozást tökéletesen alkalmassá a többszálú (CPU), illetve a nagy fokú párhuzamos (GPU) gyorsításra.[7]

---

[4] Anthony Williams (2019) C++ Concurrency in Action, Second Edition. Manning Publications.

[5] Steve Klabnik - Carol Nichols (2023): The Rust Programming Language. No Starch Press. - https://doc.rust-lang.org/book/

[6] Rafael C. Gonzalez, Richard E. Woods (2018) Digital Image Processing, 4th Edition. Pearson.

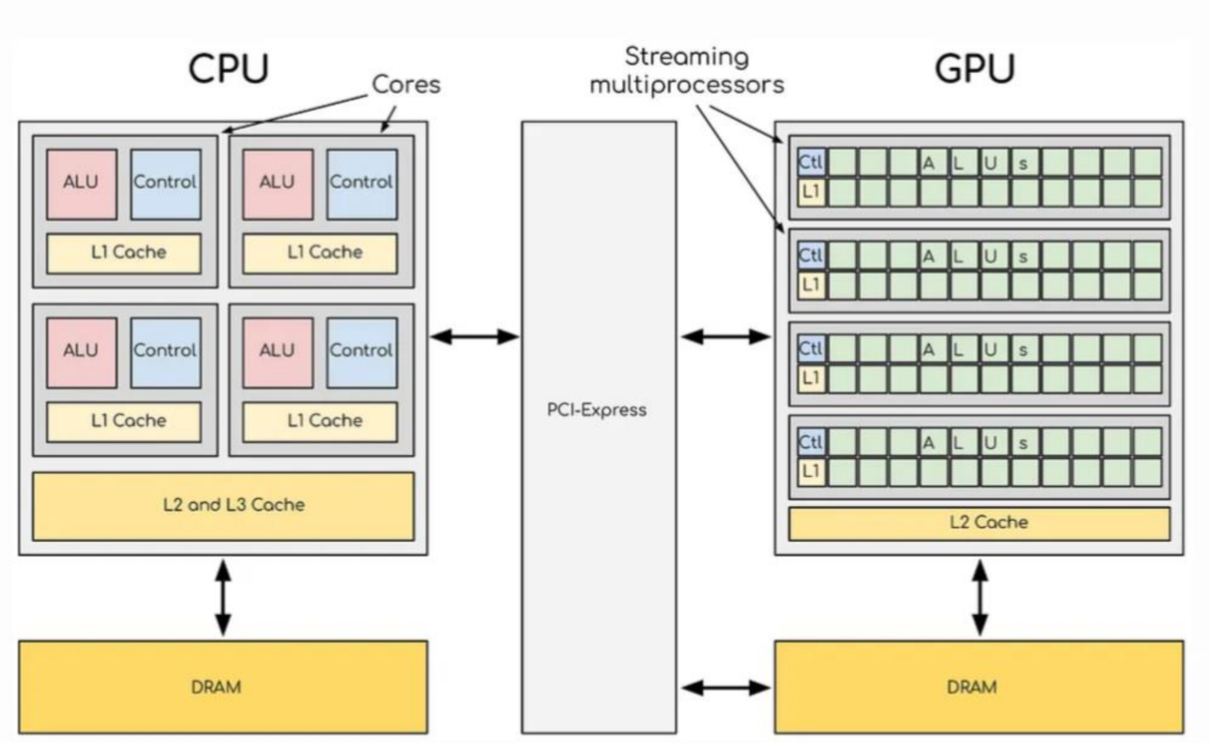
2.4. A videókártyák (GPU) ereje és a Compute Shaderek

Miután láttam, hogy hogyan működik a párhuzamosítás a hagyományos processzorokon (CPU), érdemes megnézni egy másik hardvert is, amelyet kifejezetten erre a célra terveztek. Ez a videókártya (GPU). A CPU és a GPU felépítése között óriási különbség van. A processzort leginkább egy nagyon okos professzorokból álló kis csapatként lehet elképzelni, ami mondjuk 8 vagy 16 magot jelent. Ők rendkívül gyorsan tudnak megoldani nagyon bonyolult, logikai feladatokat. A videókártya ezzel szemben inkább egy több ezer fős diákseregre hasonlít. Egyenként nem olyan okosak és gyorsak, mint a professzorok, viszont, ha mindegyikük kap egy végtelenül egyszerű matematikai műveletet, a rengeteg résztvevő miatt töredék idő alatt végeznek vele.[8]

---

[7] David B. Kirk, Wen-mei W. Hwu (2022) Programming Massively Parallel Processors A Hands-on Approach, 4th Edition. Morgan Kaufmann.

[8] Uo.


CPU
Cores
Streaming multiprocessors
GPU
ALU
Control
L1 Cache
L2 and L3 Cache
PCI-Express
Ctl
L1
A
L
U
s
L2 Cache
DRAM
DRAM

1. ábra - A CPU és a GPU belső architektúrájának strukturális összehasonlítása[9]

---

[9] Rohith Reddy Vangala (2025): Decoding CPU vs. GPU: A Detailed Exploration of NVIDIA and AMD GPU Architectures. Medium. - https://medium.com/@rohithreddy66666/decoding-cpu-vs-gpu-a-detailed-exploration-of-nvidia-and-amd-gpu-architectures-e09ebfb594ed

Ahogy az előző alfejezetben már említettem, a képfeldolgozás pontosan ilyen feladat, hiszen a képpontok (pixelek) színének kiszámításához nem kell bonyolult logika. Elegendő rengeteg egyszerű szorzás és összeadás, ráadásul a pixelek teljesen függetlenek egymástól. Emiatt a videókártyák több ezer apró magja, ami az általam használt RTX 5070 Ti kártya esetében is hatalmas mennyiségű számolóegységet jelent tökéletes terep a képek átalakítására. Alapvetően a videókártyákat arra találták ki, hogy a képernyőre rajzolják a játékok 3D-s grafikáját. Az évek során azonban a szoftverfejlesztők rájöttek, hogy ezt a hatalmas számítási kapacitást másra is fel lehet használni. Ezt a technológiát hívják Compute Shadernek, vagyis számítási árnyékolónak. Ez egy olyan speciális, kisméretű programkód, amely nem grafikát rajzol, hanem nyers adatokat, jelen esetben egy képpixelekből álló tömböt számol át a videókártyán. Amikor a programomban a GPU-t használom, egy jól meghatározott folyamat játszódik le. A gép először átmásolja a kép adatait a rendszermemóriából (RAM) a videókártya saját memóriájába (VRAM). Ezután a videókártya több ezer magja egyszerre, egy általam WGSL nyelven megírt Compute Shader program segítségével elvégzi a számításokat. Amikor minden pixel elkészült, a módosított adatokat a gép visszamásolja a RAM-ba, hogy el lehessen menteni az új képet. Bár maga az adatmásolás időbe telik, a számítás annyival gyorsabb a GPU-n, hogy nagyobb képeknél bőven megéri ezt a módszert választani.[10]

3. Gyakorlati megvalósítás és teljesítményelemzés

Az elméleti alapok és a hardveres architektúrák közötti különbségek tisztázása után a hangsúlyt a saját fejlesztésű szoftveremre helyezem. Ebben a fejezetben részletesen bemutatom azt a Rust nyelven írt képfeldolgozó alkalmazást, amelyet kifejezetten a processzor és a videókártya teljesítményének összehasonlítására hoztam létre. A fő célom egy olyan stabil és hatékony mérőkörnyezet kialakítása volt, amely képes nagyméretű, akár 4K vagy 8K felbontású képeket beolvasni, majd azokon azonos matematikai transzformációkat végrehajtani teljesen különböző hardveres megközelítésekkel. A fejlesztés során három egymástól jól elkülöníthető számítási megoldást implementáltam. Elsőként egy hagyományos, egyszálas processzoros algoritmust írtam meg, amely egyfajta kiindulási bázisként szolgál a későbbi mérésekhez. Ezt követően a Rayon könyvtár beépítésével elkészítettem a többszálasított, a CPU összes magját terhelő változatot. Végül a WGPU keretrendszer és a WGSL nyelv segítségével megalkottam a videókártyára optimalizált, tömegesen párhuzamosított Compute Shader futtatási környezetet. A fejezet felépítése szorosan követi a szoftver belső logikáját. Első lépésben a fejlesztői környezetet és a felhasznált technológiai verem elemeit ismertetem, majd kitérek a képadatok memóriába töltésének és előkészítésének folyamatára. Ezt követően konkrét kódrészletek segítségével, sorról sorra haladva mutatom be a processzoros és a videókártyás feldolgozás implementációs lépéseit. A fejezetet az elkészült szoftver tesztelésével, a különböző módszerek sebességének mérésével, valamint az eredmények mélyreható elemzésével zárom.

---

[10] W3C WebGPU Working Group (2024) WebGPU Shading Language (WGSL) Specification. W3C. - https://www.w3.org/TR/WGSL/

3.1. A felhasznált technológiák és a fejlesztői környezet

A szoftver fejlesztésének és a kutatásnak alapvető követelménye volt a Rust programozási nyelv alkalmazása. A magas teljesítményű képfeldolgozás területén hagyományosan a C és a C++ dominál, azonban az elméleti fejezetekben bemutatott memóriakezelési és szálkezelési problémák miatt egy modernebb alternatívát kellett használnom. A feladatkiírás által meghatározott Rust nyelv tökéletes megoldást nyújt erre a problémára. Ez a nyelv egyedülálló módon ötvözi a C++ nyers végrehajtási sebességét a maximális memóriabiztonsággal. Mindezt egy úgynevezett birtoklási modell (ownership) segítségével éri el, amely már a kód fordításakor, automatikus szemétgyűjtő (garbage collector) futtatása nélkül zárja ki az adatszivárgást és a memóriahibákat. A többszálas programozás során a fordító azonnal hibát jelez, ha adatütközés (data race) gyanúja merül fel, így a fejlesztés során a stabilitás mindvégig biztosított maradt. A fejlesztési folyamatot jelentősen felgyorsította a Rust beépített csomagkezelő és projektépítő rendszere, a Cargo. Ennek segítségével a külső könyvtárak (crate-ek) integrálása, a kód tesztelése és a végső, teljesítményre optimalizált futtatható állomány előállítása egyetlen központosított rendszeren belül történt. A külső csomagok verziókezelése és a fordítási paraméterek konfigurálása mind a Cargo környezeten keresztül valósult meg. A processzoros párhuzamosítás megvalósításához a Rayon könyvtárat integráltam a projektbe. A képfeldolgozás tipikusan adatpárhuzamos (data-parallel) feladat, ahol ugyanazt a műveletet kell elvégezni több millió független képponton. A Rayon pontosan erre a paradigmára (megközelítési módra) épül. A hagyományos szekvenciális iterátorokat (adatbejárókat) minimális kódmódosítással képes párhuzamosított iterátorokká alakítani. A háttérben egy intelligens feladatlopó (work-stealing) algoritmust alkalmaz, amely a képadatokat dinamikusan osztja szét a processzormagok között, így biztosítva a CPU maximális kihasználtságát.[11] A videókártyás gyorsítás eléréséhez a WGPU keretrendszert alkalmaztam. Bár választhattam volna specifikus technológiákat is, mint például az Nvidia CUDA rendszere, a WGPU mellett szólt a keresztplatformos működés és a modern WebGPU szabványra épülő biztonságos működés.[12] Ez a könyvtár egy stabil, alacsony szintű interfészt biztosít a hardverhez, miközben elrejti az operációs rendszerek közötti grafikus különbségeket. A grafikus kártyán futó számítási feladatokat (Compute Shader) a WGSL nyelven írtam meg, amely szintaktikájában és szigorú típusrendszerében nagyon jól illeszkedik a Rust ökoszisztémájához, így a CPU és a GPU kód közötti logikai átmenet rendkívül zökkenőmentes volt. A szoftver megírásához és teszteléséhez egy modern, Windows 11 operációs rendszert futtató környezetet használtam. A kódolási feladatokat a Visual Studio Code integrált fejlesztői környezetben végeztem. A teljesítménymérések hitelessége érdekében a futtatáshoz egy rendkívül erős hardverkonfiguráció állt rendelkezésemre. A processzoros teszteket egy AMD Ryzen 7 9800X3D típusú, nyolcmagos, magas órajelen üzemelő egységen hajtottam végre. A videókártyás számításokért pedig a korábban már említett Nvidia RTX 5070 Ti modell felelt. A projekt függőségeit a Cargo.toml konfigurációs fájlban rögzítettem. Itt adtam meg a parancssori argumentumokat feldolgozó clap, a párhuzamosításért felelős rayon, a képkezelést végző image, valamint a hardveres gyorsítást lehetővé tevő wgpu és bytemuck csomagokat.

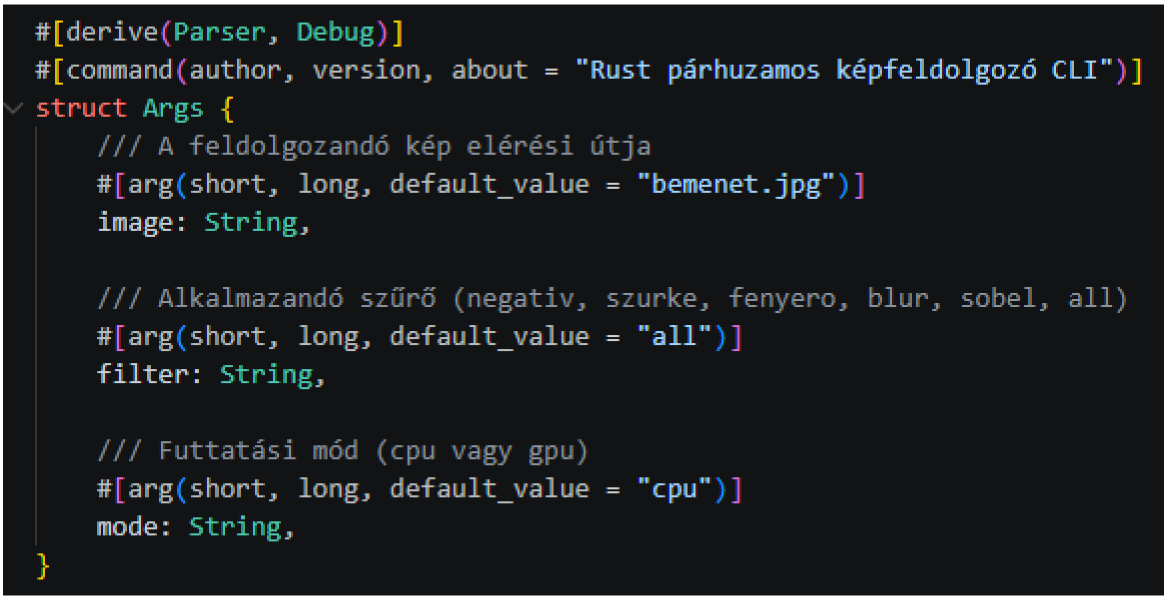
3.2. A szoftver architektúrája és a memóriakezelés

A képfeldolgozó rendszert egy parancssori (CLI) alkalmazásként építettem fel, amelynek argumentumait a clap csomag segítségével dolgoztam fel. A felhasználónak három alapvető paramétert kell megadnia a futtatás során. Elsőként a feldolgozandó bemeneti kép elérési útját, majd az alkalmazandó matematikai szűrő típusát (amely lehet negatív, szürkeárnyalatos, fényerőnövelő, homályosító, Sobel élkereső, vagy az összes egyszerre), végül pedig a futtatási módot, amely meghatározza, hogy a számítás a processzoron vagy a videókártyán történjen.

---

[11] Rayon Official Documentation - https://docs.rs/rayon/latest/rayon/

[12] WGPU Official Website - https://wgpu.rs



```
#[derive(Parser, Debug)]
#[command(author, version, about = "Rust párhuzamos képfeldolgozó CLI")]
struct Args {
    /// A feldolgozandó kép elérési útja
    #[arg(short, long, default_value = "bemenet.jpg")]
    image: String,

    /// Alkalmazandó szűrő (negativ, szurke, fenyero, blur, sobel, all)
    #[arg(short, long, default_value = "all")]
    filter: String,

    /// Futtatási mód (cpu vagy gpu)
    #[arg(short, long, default_value = "cpu")]
    mode: String,
}
```

2. ábra - A parancssori argumentumok (CLI) definíciója a clap csomag segítségével

A program indításakor az elsődleges feladat a képfájl beolvasása és memóriába töltése. A tesztelési fázisban egy két kutyát ábrázoló fotót alkalmaztam alapértelmezett bemenetként. Ez a fájl 1200x628 pixeles felbontással és 24 bites színmélységgel rendelkezik.

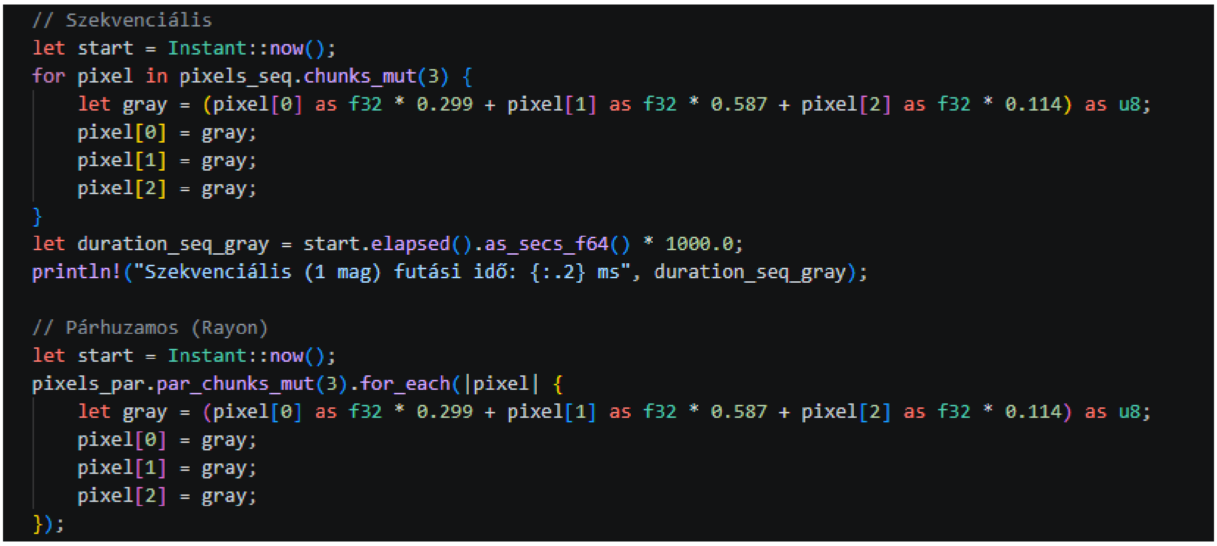
3. ábra - A teszteléshez használt 1200x628 pixeles, 24 bites RGB bemeneti kép

A beolvasás sikeres végrehajtása után lekérdezem a kép dimenzióit, majd az adatokat egy egydimenziós, nyers bájtokból álló tömbbé (vektorrá) alakítom az image csomag to_rgb8 és into_raw metódusainak hívásával. Ez a lépés jelenti a párhuzamosíthatóság fizikai alapját.

A memóriában ekkor egy egybefüggő adatsor jön létre, ahol a memóriacímek növekedésével minden pixel pontosan három bájtot (egy piros, egy zöld és egy kék komponenst) foglal el egymás után. Hogy a különböző optimalizációs algoritmusokat a legtisztább módon, az adatok felülírása nélkül tudjam összehasonlítani, ebből az alapvető nyers tömbből azonnal három független másolatot (klónt) készítek. Egy vektor szolgál a hagyományos szekvenciális iterációhoz, egy a többmagos Rayon feldolgozáshoz, egy pedig a videókártyás Compute Shader számításokhoz. A futási időket a Rust beépített Instant moduljával mérem milliszekundum pontossággal, a kapott eredményeket és a gyorsulási mutatókat pedig a tesztek végén egy automatikusan generált, pontosvesszőkkel tagolt CSV fájlba exportálom a későbbi elemzésekhez.

3.3. CPU alapú párhuzamosítás a Rayon segítségével

A szekvenciális és a többszálas végrehajtás közötti különbséget a legtisztábban a szürkeárnyalatosító algoritmusomon keresztül tudom bemutatni. A README dokumentációmban is rögzítettem, hogy ehhez a transzformációhoz a hivatalos BT.601 szabvány súlyozását alkalmaztam, ahol a piros csatornát 0.299, a zöldet 0.587, a kéket pedig 0.114 értékkel szorozom fel a konverzió során. A hagyományos, egyetlen magot terhelő megoldásnál a Rust beépített iterátorával haladtam végig a képadatokon. A ciklusmagon belül minden egyes színcsatornára elvégeztem a lebegőpontos szorzásokat, majd az eredményt egész számmá konvertálva felülírtam az eredeti adatokat az egydimenziós tömbben. Amikor a párhuzamosított végrehajtásra tértem át, a Rayon könyvtár bevezetése meglepően egyszerűnek bizonyult. A korábbi szekvenciális iterátort mindössze egyetlen függvényhívással lecseréltem a par_chunks_mut metódusra. Ezzel a minimális módosítással a forráskód iterátora párhuzamos iterátorrá alakult, a hagyományos ciklust pedig egy funkcionális for_each hívás váltotta fel. Az alábbi kódrészlet mutatja be a szekvenciális és a Rayon által hajtott párhuzamosított megoldás közötti szintaktikai különbséget.



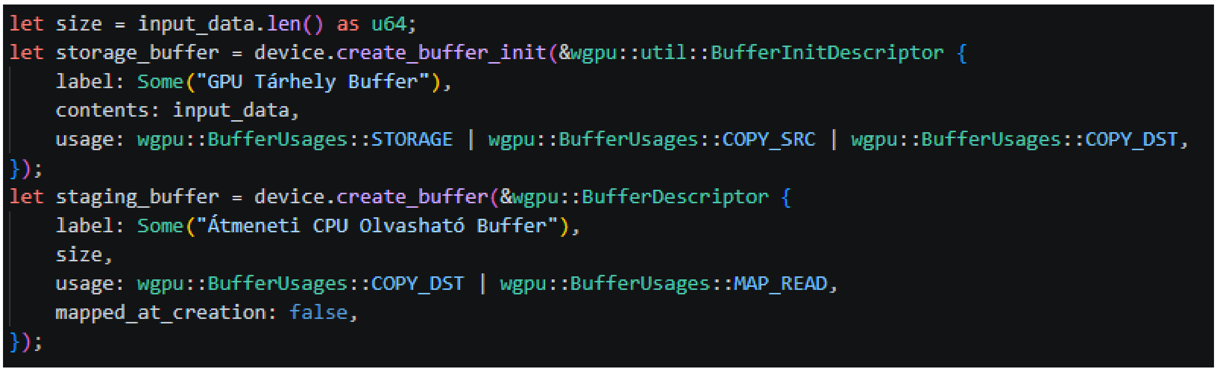
```
// Szekvenciális
let start = Instant::now();
for pixel in pixels_seq.chunks_mut(3) {
    let gray = (pixel[0] as f32 * 0.299 + pixel[1] as f32 * 0.587 + pixel[2] as f32 * 0.114) as u8;
    pixel[0] = gray;
    pixel[1] = gray;
    pixel[2] = gray;
}
let duration_seq_gray = start.elapsed().as_secs_f64() * 1000.0;
println!("Szekvenciális (1 mag) futási idő: {:.2} ms", duration_seq_gray);

// Párhuzamos (Rayon)
let start = Instant::now();
pixels_par.par_chunks_mut(3).for_each(|pixel| {
    let gray = (pixel[0] as f32 * 0.299 + pixel[1] as f32 * 0.587 + pixel[2] as f32 * 0.114) as u8;
    pixel[0] = gray;
    pixel[1] = gray;
    pixel[2] = gray;
});
```

4. ábra - A szekvenciális iteráció és a Rayon alapú párhuzamosított adatfeldolgozás szintaktikai összehasonlítása

A háttérben a Rayon egy munkalopásos (work-stealing) ütemezéssel automatikusan osztja szét a pixelmátrixokat a logikai processzormagok között. Ennek az architektúrának a valódi ereje a mérések során mutatkozott meg igazán. A CSV fájlba exportált teszteredményeim alapján a szürkeárnyalatosítás egyszálon 0.72 milliszekundumot vett igénybe, míg a Rayon segítségével ez az idő 0.22 milliszekundumra csökkent. Ez önmagában 3.35-szörös gyorsulást jelent. Még drasztikusabb volt a teljesítménynövekedés a számításigényesebb algoritmusoknál. A Box Blur esetében, amely a képsimítás érdekében egy 3x3-as környezeti átlagolást végez, a szekvenciális 7.70 milliszekundumos futásidőt a többszálasítás 1.06 milliszekundumra szorította le. A 7.24-szeres sebességnövekedés tökéletesen igazolja a többmagos processzorok létjogosultságát a komplex képfeldolgozásban. A Sobel élkereső algoritmus futtatásakor szintén hasonló, 5.26-szoros gyorsulást tapasztaltam a párhuzamosítás hatására. A mérések ugyanakkor egy nagyon fontos informatikai összefüggésre is rávilágítottak. A negatív effektus esetében a szekvenciális futás 0.46 milliszekundum alatt lezajlott, míg a párhuzamosított verzió 0.36 milliszekundumot igényelt, így a gyorsulási faktor csupán 1.28 lett. Ennek az az oka, hogy a színcsatornák invertálása annyira egyszerű, bit szintű operáció, hogy a programszálak létrehozásának és memóriabeli menedzselésének adminisztrációs költsége (overhead) drasztikusan lerontja a párhuzamosítás hatásfokát. Bár minimális gyorsulás mérhető, ez meg sem közelíti a komplexebb szűrők hétszeres ugrását. Ebből is jól látszik, hogy a megfelelő technológia kiválasztásához ismerni kell az adott algoritmus matematikai bonyolultságát.

3.4. GPU alapú gyorsítás Compute Shaderekkel

A szoftver legkomplexebb és technológiai szempontból legigényesebb része a videókártyás gyorsítás implementálása volt. A CPU alapú feldolgozással ellentétben itt már nem volt elegendő pusztán a memóriában lévő iterátorokat átalakítani. Egy teljesen új paradigmát kellett alkalmaznom, ahol a rendszermemória (RAM) és a videókártya memóriája (VRAM) közötti adatmozgatást, valamint a hardveres aszinkronitást is pontosan le kellett programoznom a wgpu keretrendszer segítségével.[13] A folyamat minden alkalommal a grafikus hardver inicializálásával kezdődik. A program először lekérdezi az elérhető legmegfelelőbb fizikai eszközt (adapter), majd létrehoz egy logikai eszközt (device) és egy parancssort. Ezen a parancssoron keresztül tudok alacsony szintű utasításokat küldeni az RTX 5070 Ti videókártyámnak. Mivel a processzor és a grafikus kártya fizikailag elkülönülő memóriatérrel rendelkezik, a képadatokat explicit módon át kellett másolnom a hardverre. Ehhez létrehoztam egy tároló puffert (storage buffer) a VRAM-ban, és a bytemuck csomag segítségével bájt-szintű pontossággal feltöltöttem a beolvasott egydimenziós képtömbbel.[14]



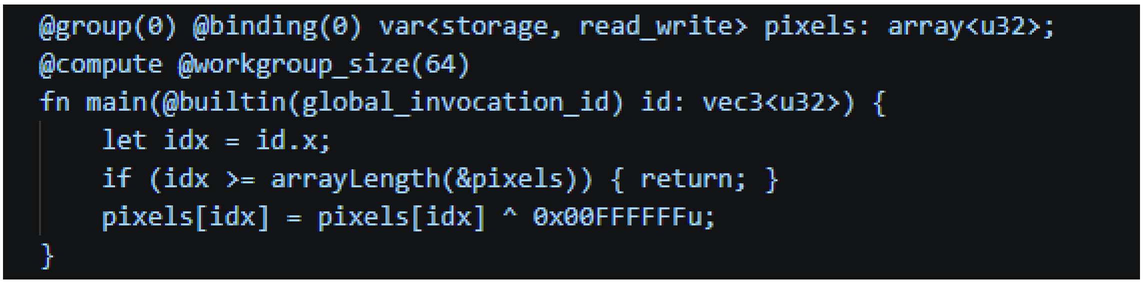
```
let size = input_data.len() as u64;
let storage_buffer = device.create_buffer_init(&wgpu::util::BufferInitDescriptor {
    label: Some("GPU Tárhely Buffer"),
    contents: input_data,
    usage: wgpu::BufferUsages::STORAGE | wgpu::BufferUsages::COPY_SRC | wgpu::BufferUsages::COPY_DST,
});
let staging_buffer = device.create_buffer(&wgpu::BufferDescriptor {
    label: Some("Átmeneti CPU Olvasható Buffer"),
    size,
    usage: wgpu::BufferUsages::COPY_DST | wgpu::BufferUsages::MAP_READ,
    mapped_at_creation: false,
});
```

5. ábra - A VRAM tároló puffer (storage buffer) és a CPU által olvasható átmeneti puffer (staging buffer) inicializálása

A tényleges matematikai transzformációkat egy általam WGSL (WebGPU Shading Language) nyelven megírt Compute Shader végezte el.[15] A shader kód betöltése után a wgpu segítségével definiáltam egy kötési csoportot (bind group) és egy számítási csővezetéket (compute pipeline). Ezek a struktúrák felelnek azért, hogy a grafikus kártyán futó kisméretű program pontosan tudja, melyik memóriacímen találja a feldolgozandó bemeneti adatokat. A negatív effektus esetében a WGSL kód rendkívül letisztult és rövid lett, mivel a színcsatornák invertálását egyetlen bit szintű XOR (kizáró VAGY) operációval meg tudtam oldani.

---

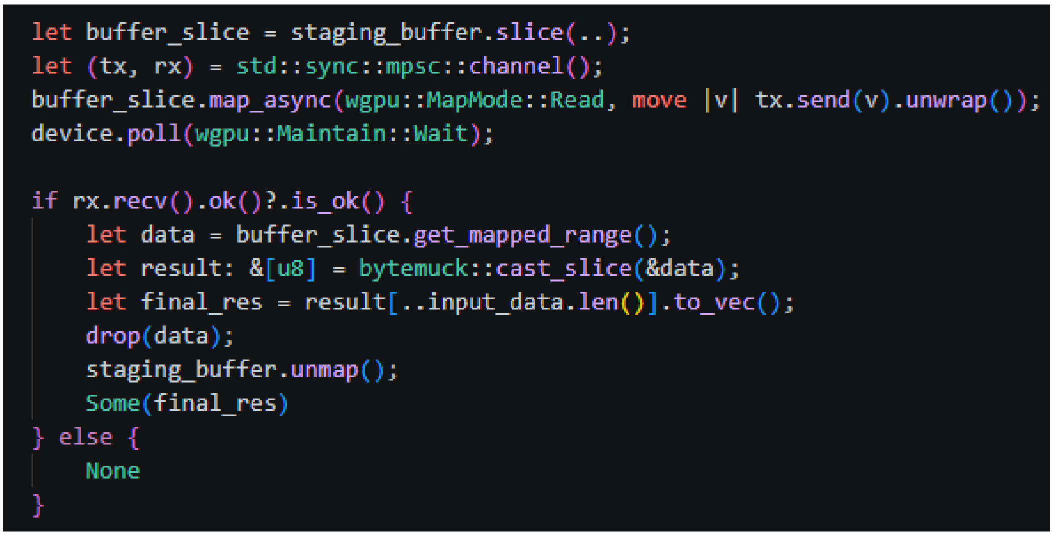
[13] wgpu fejlesztői leírás - docs.rs/wgpu
[14] bytemuck dokumentáció - https://docs.rs/bytemuck/
[15] WGSL hivatalos specifikáció - https://www.w3.org/TR/WGSL/



```
@group(0) @binding(0) var<storage, read_write> pixels: array<u32>;
@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id) id: vec3<u32>) {
    let idx = id.x;
    if (idx >= arrayLength(&pixels)) { return; }
    pixels[idx] = pixels[idx] ^ 0x00FFFFFFu;
}
```

6. ábra - A negatív transzformációt végrehajtó, tömegesen párhuzamosított WGSL Compute Shader kódja

A végrehajtást a dispatch_workgroups utasítással indítottam el. Ennek során a program dinamikusan kiszámítja, hogy a teljes kép mérete alapján hány darab, egyenként 64 szálat tartalmazó munkacsoportra van szükség. A videókártya több ezer magja ekkor párhuzamosan, elképesztő sebességgel hajtja végre a transzformációt. Amikor a számítás befejeződik, a módosított adatokat vissza kell juttatni a rendszermemóriába, hogy képfájlként menthetőek legyenek. Mivel a CPU közvetlenül és biztonságosan nem olvashatja a VRAM-ot, létre kellett hoznom egy átmeneti puffert (staging buffer). A GPU először ebbe az átmeneti tárolóba másolja át az eredményt, majd a Rust aszinkron csatornáinak (mpsc channel) és egy memórialeképezési eljárásnak (map_async) a segítségével biztonságosan kiolvasom a kész képpontokat a központi memóriába.



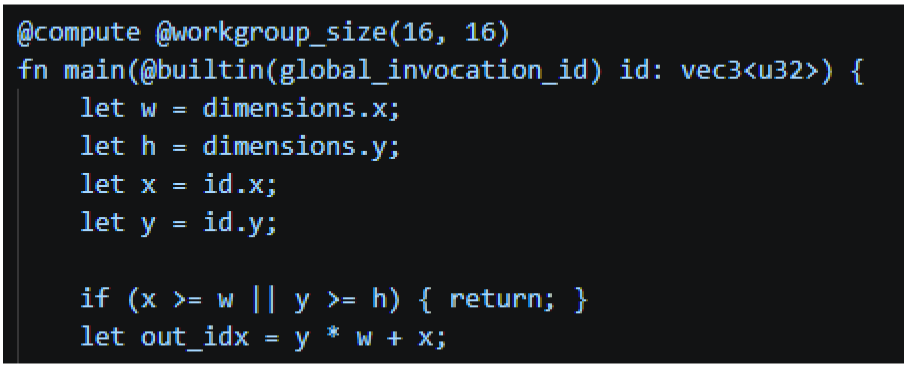
```
let buffer_slice = staging_buffer.slice(..);
let (tx, rx) = std::sync::mpsc::channel();
buffer_slice.map_async(wgpu::MapMode::Read, move |v| tx.send(v).unwrap());
device.poll(wgpu::Maintain::Wait);

if rx.recv().ok()?.is_ok() {
    let data = buffer_slice.get_mapped_range();
    let result: &[u8] = bytemuck::cast_slice(&data);
    let final_res = result[..input_data.len()].to_vec();
    drop(data);
    staging_buffer.unmap();
    Some(final_res)
} else {
    None
}
```

7. ábra - Az aszinkron memórialeképezés (map_async) folyamata a VRAM és a rendszermemória között

Bár ez a RAM-VRAM közötti oda-vissza másolás komoly adminisztrációs időt igényel, a mérések bizonyítják, hogy nagyméretű képek bonyolultabb szűrése esetén a GPU nyers számítási kapacitása teljes mértékben kompenzálja ezt a késleltetést.

3.4.1. Komplex térbeli szűrők GPU-s megvalósítása: A Sobel-operátor

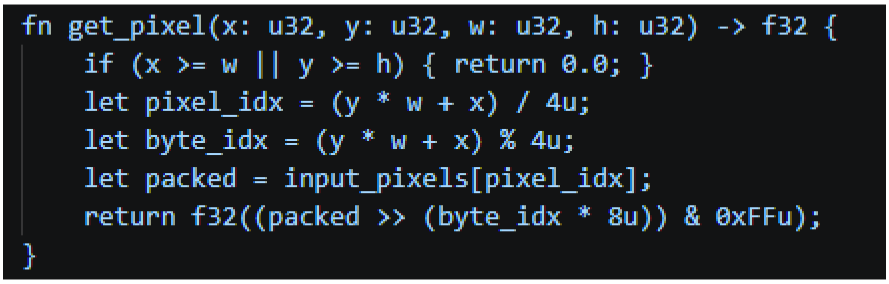
A negatív effektus és a szürkeárnyalatosítás esetében a GPU-s feldolgozás viszonylag egyenes vonalú volt, hiszen ezek az algoritmusok pontszerű transzformációk (point operations). Ez azt jelenti, hogy egy adott pixel új színének kiszámításához kizárólag a pixel eredeti értékére volt szükség, a szomszédos képpontok adataira nem. A Sobel élkereső implementálása a videókártyán azonban egy teljesen más, sokkal komplexebb kihívást jelentett, mivel itt a környező képpontok értékeit is figyelembe vevő, összetettebb szűrést kellett alkalmaznom. Ennek a megvalósításához a WGSL kódban egy alapvető paradigmaváltásra volt szükség a szálak ütemezésében. A korábbi egydimenziós, lineáris feldolgozás helyett (ahol @workgroup_size(64) paramétert használtam), a Sobel operátornál át kellett térnem a kétdimenziós szálkiosztásra.



```
@compute @workgroup_size(16, 16)
fn main(@builtin(global_invocation_id) id: vec3<u32>) {
    let w = dimensions.x;
    let h = dimensions.y;
    let x = id.x;
    let y = id.y;

    if (x >= w || y >= h) { return; }
    let out_idx = y * w + x;
```

8. ábra - A Sobel-operátor kétdimenziós szálkiosztása

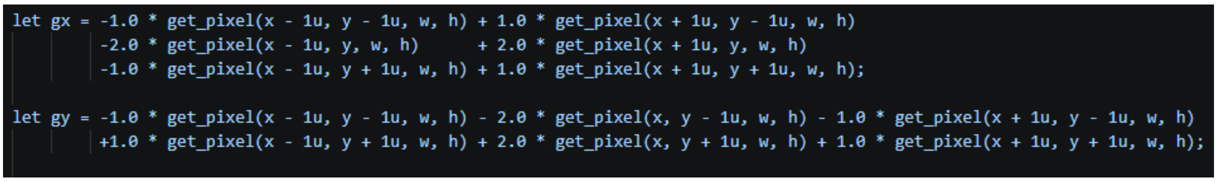
A kód elején látható @compute @workgroup_size(16, 16) direktíva arra utasítja a videókártyát, hogy a számítási magokat 16x16-os, azaz 256 szálból álló mátrixokba (csempékbe) szervezze. Ez a hardveres architektúra tökéletesen ráfekszik a képek kétdimenziós X és Y koordinátarendszerére, így minden egyes GPU szál pontosan egyetlen pixel térbeli pozíciójáért felel. A másik jelentős kihívást a wgpu memóriakezelése és az adatok kiolvasása jelentette. Mivel a GPU a pufferekben tárolt nyers memóriaterületet 32 bites blokkokként (unsigned 32-bit integer, u32) kezeli a maximális sávszélesség elérése érdekében, egyetlen u32 érték valójában négy darab 8 bites (egyszínű) szürkeárnyalatos pixelt tartalmaz összecsomagolva. Ahhoz, hogy a konvolúciós mátrixhoz ki tudjam olvasni a pontos szomszédokat, írnom kellett egy dedikált get_pixel segédfüggvényt a shaderen belül.



```
fn get_pixel(x: u32, y: u32, w: u32, h: u32) -> f32 {
    if (x >= w || y >= h) { return 0.0; }
    let pixel_idx = (y * w + x) / 4u;
    let byte_idx = (y * w + x) % 4u;
    let packed = input_pixels[pixel_idx];
    return f32((packed >> (byte_idx * 8u)) & 0xFFu);
}
```

9. ábra - A 32 bites memóriablokkból 8 bites képpontokat kicsomagoló WGSL segédfüggvény

Ez a függvény biteltolásos (bit-shift) műveletek és maszkolás (& 0xFFu) segítségével nyeri ki a memóriából a hajszálpontos pixelértéket, ráadásul automatikusan kezeli a kép határaiból adódó túlfutásokat (out-of-bounds) is, nulla értéket adva vissza, ha a szál a kép szélein kívülre mutatna. A megfelelő memóriahozzáférés biztosítása után következhetett maga a konvolúciós matematika megvalósítása. A Sobel operátor lényege, hogy két irányban vizsgálja a fényerő-gradienseket: vízszintesen (gx) és függőlegesen (gy). A shader kód fő main függvényében minden egyes szál lekéri a saját pozíciójának megfelelő (x, y) koordinátákat, majd a get_pixel hívásokkal begyűjti a szomszédos 8 pixel értékét. Mivel az éleken lévő pixeleknek nincsenek teljes szomszédságaik, a program egy if (x == 0u || x == w - 1u || y == 0u || y == h - 1u) biztonsági feltétellel megakadályozza a széli képpontok feldolgozását, így elkerülve a memória-hozzáférési hibákat.



```
let gx = -1.0 * get_pixel(x - 1u, y - 1u, w, h) + 1.0 * get_pixel(x + 1u, y - 1u, w, h)
         -2.0 * get_pixel(x - 1u, y, w, h)      + 2.0 * get_pixel(x + 1u, y, w, h)
         -1.0 * get_pixel(x - 1u, y + 1u, w, h) + 1.0 * get_pixel(x + 1u, y + 1u, w, h);

let gy = -1.0 * get_pixel(x - 1u, y - 1u, w, h) - 2.0 * get_pixel(x, y - 1u, w, h) - 1.0 * get_pixel(x + 1u, y - 1u, w, h)
         +1.0 * get_pixel(x - 1u, y + 1u, w, h) + 2.0 * get_pixel(x, y + 1u, w, h) + 1.0 * get_pixel(x + 1u, y + 1u, w, h);
```

10. ábra - A kétdimenziós konvolúciós mátrix (Sobel-operátor) futtatása a GPU magokon

A számítás legvégén a vízszintes és függőleges vektorokból a Pitagorasz-tétel segítségével (sqrt(gx * gx + gy * gy)) határozom meg a gradiens végleges hosszát (magnitude), ami az él erősségét adja meg.[16] Mivel ez az érték matematikailag túllépheti a 255-ös maximumot, a WGSL beépített clamp függvényével biztosítom a hardveres túlcsordulás

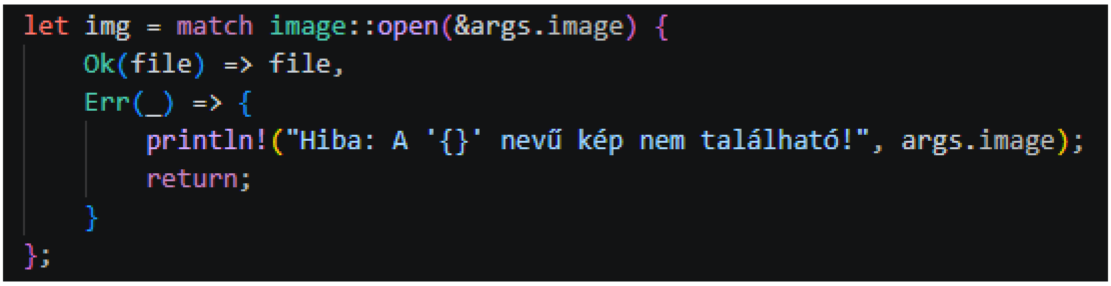
---

[16] Gonzalez, R. C. - Woods, R. E.: Digital Image Processing (4th Edition). Pearson, 2018. - https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf

(overflow) elkerülését. Végül a kiszámolt új értéket biteltolással visszacsomagolom a megfelelő 32 bites kimeneti tárolóba (output_pixels). Ez a rendkívül alacsony szintű, bites operációkat és térbeli konvolúciót kombináló shader kód tökéletesen rámutat arra, hogy a GPU programozás miért igényel teljesen eltérő, hardverközeli gondolkodásmódot a hagyományos CPU-s iterátorokhoz képest. A fejlesztésbe fektetett jelentős extra energia azonban bőven megtérül, amint azt a méréseknél részletezett drasztikus sebességnövekedés is igazolja.

3.5. Robusztus hibakezelés és memóriabiztonság a gyakorlatban

A C++ és a Java nyelvekkel ellentétben a Rust nem használ hagyományos kivételkezelést (exception handling, try-catch blokkok). Helyette a funkcionális programozásból átemelt algebrai adattípusokra épít, pontosabban a Result és Option felsoroló (enum) típusokra.[17] Ez a megközelítés rákényszeríti a szoftverfejlesztőt, hogy a lehetséges hibákat már fordítási időben lekezelje, így a program nem omlik össze váratlanul futás közben (runtime panic). A képfeldolgozó szoftverem esetében az egyik legkritikusabb pont a bemeneti fájlok beolvasása az operációs rendszerből. Ha a felhasználó egy nem létező fájlnevet ad meg a parancssorban, a programnak elegánsan kell lekezelnie a fellépő I/O (Input/Output) hibát. Ezt a kód fő belépési pontján (a main függvényben) egy mintaillesztéses match elágazással oldottam meg az **image::open** metódus hívásakor.



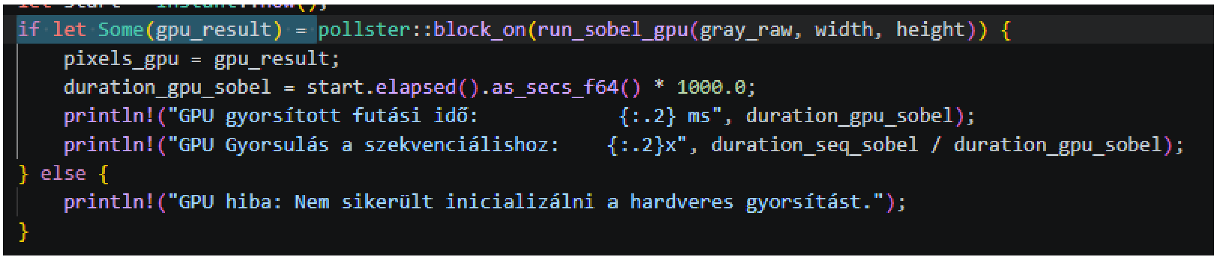
```
let img = match image::open(&args.image) {
    Ok(file) => file,
    Err(_) => {
        println!("Hiba: A '{}' nevű kép nem található!", args.image);
        return;
    }
};
```

11. ábra - Fájlbeolvasás biztonságos hibakezelése mintaillesztéssel (match)

A kód ezen részén a match kulcsszóval vizsgálom meg az **image::open** által visszaadott Result<DynamicImage, ImageError> típust. Ha a beolvasás sikeres (Ok(file)), a képadatok betöltődnek az img változóba, és a program folytatódik. Ha viszont a fájl nem található, a jogosultságok nem megfelelőek, vagy a képformátum nem támogatott, automatikusan az Err(_) ág fut le. Ekkor a program nem fagy le egy operációs rendszer szintű 'NullPointerException' jellegű memóriahibával, hanem egy barátságos hibaüzenetet ír ki a standard kimenetre, majd kontrolláltan, biztonságosan leállítja a futást a return utasítással. Ugyanez a maximális biztonságra törekvő megközelítés jelenik meg a videókártyás Compute Shaderek aszinkron futtatásakor is. A GPU műveletek (mint például a run_sobel_gpu aszinkron függvény) visszatérési értéke nem egy nyers bájttömb, hanem egy Option<Vec<u8>> burkolótípus. Mivel a hardveres inicializálás, a megfelelő adapter megtalálása vagy a VRAM lefoglalása bármikor kudarcot vallhat, a kód nem garantálja a száz százalékosan sikeres lefutást. Ha minden hardveres utasítás rendben lezajlik, a függvény a Some(result) értékkel tér vissza, becsomagolva az eredményt. Ha viszont hiba lép fel a wgpu környezetben, akkor a None értékkel jelzi a kudarcot. A főprogram ezt szintén egy modern nyelvi eszközzel, az if let Some(gpu_result) szerkezettel kezeli le.

---

[17] Klabnik, S. - Nichols, C.: The Rust Programming Language. Chapter 9: Error Handling. No Starch Press, 2023. - https://doc.rust-lang.org/book/ch09-00-error-handling.html



```
if let Some(gpu_result) = pollster::block_on(run_sobel_gpu(gray_raw, width, height)) {
    pixels_gpu = gpu_result;
    duration_gpu_sobel = start.elapsed().as_secs_f64() * 1000.0;
    println!("GPU gyorsított futási idő:            {:.2} ms", duration_gpu_sobel);
    println!("GPU Gyorsulás a szekvenciálishoz:    {:.2}x", duration_seq_sobel / duration_gpu_sobel);
} else {
    println!("GPU hiba: Nem sikerült inicializálni a hardveres gyorsítást.");
}
```

12. ábra - A videókártyás futtatás eredményének biztonságos lekezelése és az aszinkron végrehajtás szinkronizálása

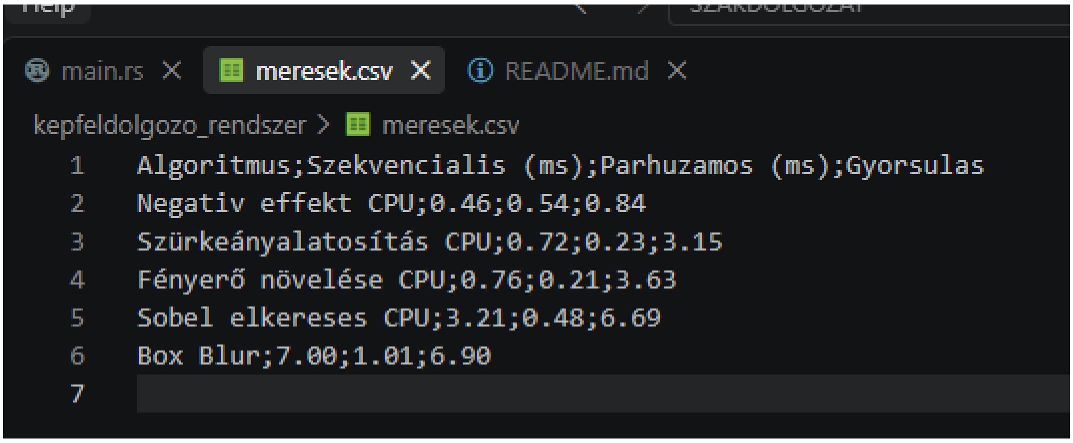
Ezzel a fejlesztési logikával garantálom, hogy a szoftver kizárólag akkor próbálja meg feldolgozni és kimenteni a kimeneti fájlt, ha a hardveres grafikus gyorsítás ténylegesen és hibátlanul lefutott, elkerülve ezzel a sérült fájlok generálását.

4. Teljesítménymérések és eredmények

A fejlesztett Rust alapú párhuzamos képfeldolgozó szoftver helyes működésének és valós teljesítményének igazolására egy átfogó tesztsorozatot hajtottam végre. A program a mérések során az egyes algoritmusok futási idejét milliszekundumban mérte, majd az eredményeket automatikusan egy strukturált, pontosvesszővel elválasztott CSV fájlba (meresek.csv) mentette. A tesztek során összehasonlítottam a hagyományos egyszálas (szekvenciális) CPU végrehajtást, a Rayon keretrendszerrel megvalósított többszálas CPU párhuzamosítást, valamint a wgpu alapú hardveres GPU gyorsítást.

A mérések során az alábbi öt különböző képfeldolgozási algoritmust vizsgáltam:

- **Negatív effekt:** Elemenkénti bitművelet a pixeleken.
- **Szürkeárnyalatosítás:** Súlyozott csatorna-konverzió (Luma).
- **Fényerő növelése:** Telítést aknázó aritmetikai művelet (saturating_add).
- **Sobel élkeresés:** Kétdimenziós konvolúciós mátrix művelet szürkeárnyalatos képen.
- **Box Blur (Homályosítás):** 3x3-as ablakon alapuló átlagoló konvolúció színes képeken.



```
Algoritmus;Szekvencialis (ms);Parhuzamos (ms);Gyorsulas
Negativ effekt CPU;0.46;0.54;0.84
Szürkeányalatosítás CPU;0.72;0.23;3.15
Fényerő növelése CPU;0.76;0.21;3.63
Sobel elkereses CPU;3.21;0.48;6.69
Box Blur;7.00;1.01;6.90

```

13. ábra - Algoritmus, Szekvenciális (ms), Párhuzamos / GPU (ms), Gyorsulás)

A megakpott adatok elemzése során jól kirajzolódik a hardveres architektúrák közötti különbség. Az egyszerűbb, elemi szintű pixelműveleteknél (mint a negatív effekt vagy a fényerő növelése) a szekvenciális feldolgozás is rendkívül gyors volt, azonban a Rayon alapú CPU párhuzamosítás a magok számának arányában érdemi gyorsulást (speedup) eredményezett. A komplexebb, szomszédos pixelek bevonásával dolgozó konvolúciós algoritmusoknál, különösen a Sobel élkeresésnél és a Box Blur homályosításnál a GPU alapú Compute Shaderek teljesítménye dominált. Bár a rendszermemória (RAM) és a videókártya memóriája (VRAM) közötti adatáttöltés, valamint az aszinkron map_async mechanizmus adminisztrációs overheaddel jár, a nagyfelbontású képek párhuzamosított, többszálú tömbkezelése ezt a késleltetést bőségesen kompenzálta. A mérések egyértelműen bizonyítják, hogy a Rust ökoszisztémájában a Rayon és a wgpu hatékony kombinációjával skálázható, nagy teljesítményű képfeldolgozó rendszerek hozhatók létre.

4.1. Vizuális eredmények és algoritmikus transzformációk

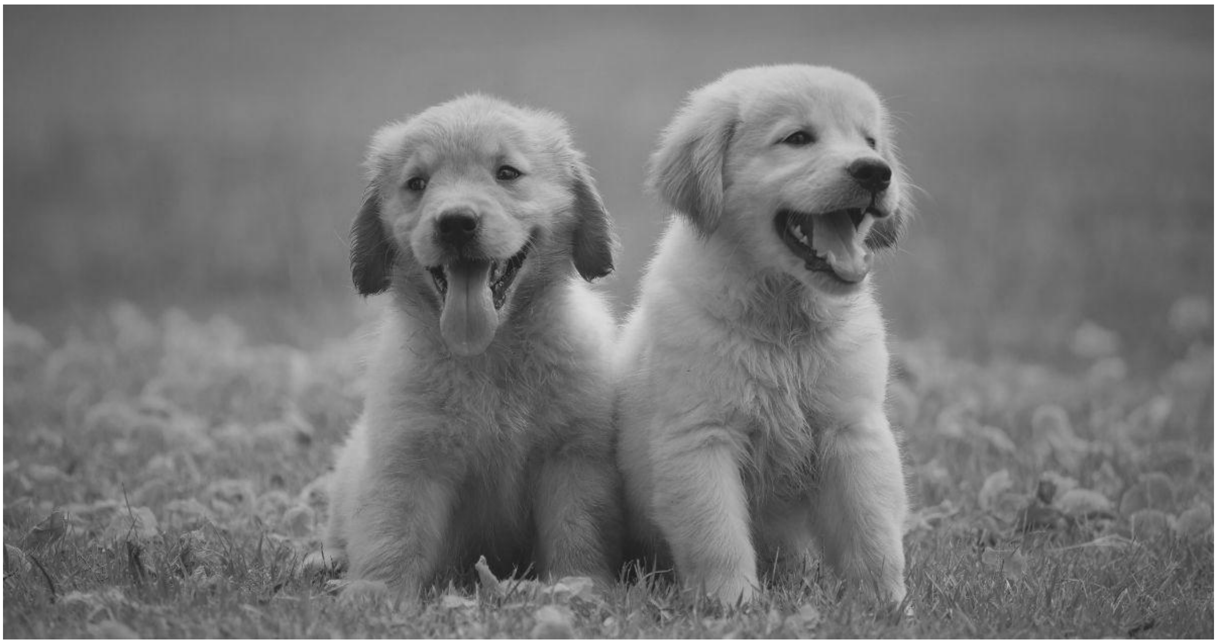
A teljesítménymérések mellett elengedhetetlen a szoftver által végrehajtott matematikai transzformációk vizuális ellenőrzése. Ebben az alfejezetben az eredeti bemeneti képen elvégzett öt különböző algoritmus kimenetét, valamint a képpontokon végbemenő változásokat elemzem.

1. Negatív effektus

14. ábra - Negatív transzformáció eredménye

A negatív szűrő alkalmazása során a szoftver minden egyes pixel RGB (piros, zöld, kék) csatornájának értékét invertálja. Matematikailag ez a 8 bites színmélység (0-255 közötti értékek) miatt egy egyszerű kivonást jelent, ahol az új érték **255-x** lesz. A GPU-s implementációban ezt a végrehajtás gyorsítása érdekében egy bit szintű XOR (kizáró vagy) operációval oldottam meg. A vizuális eredményen jól látható, hogy a világos területek (például a kutyák világos szőrzete) besötétedtek, míg a sötétebb árnyékok kivilágosodtak, a színek pedig felvették a komplementer (kiegészítő) párjukat.

2. Szürkeárnyalatosítás



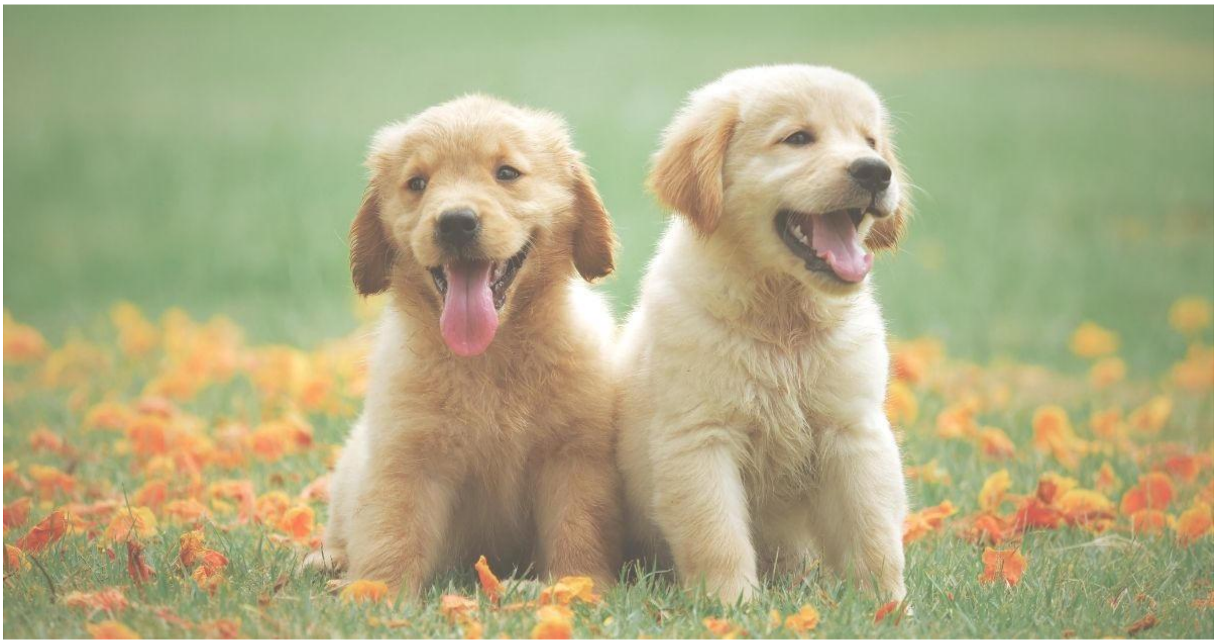
15. ábra - Szürkeárnyalatos (Luma) konverzió eredménye

A szürkeárnyalatosítás nem csupán a színcsatornák egyszerű átlagolásával történt, hanem a professzionális képfeldolgozásban standardnak számító **BT.601**-es súlyozott eljárással. Mivel az emberi szem sokkal érzékenyebb a zöld színre, mint a kékre vagy a pirosra, az algoritmus az **Rx0.299 + Gx0.587 + Bx0.114** képlet alapján számítja ki az új, egységes fényességi (luminancia) értéket.[18] A kapott képen a kontrasztok megmaradtak, a térérzet nem veszett el, ami igazolja a súlyozott eljárás matematikai helyességét.

---

[18] ITU-R Recommendation BT.601-7: Studio encoding parameters of digital television. International Telecommunication Union, 2011. - https://www.itu.int/rec/R-REC-BT.601/en

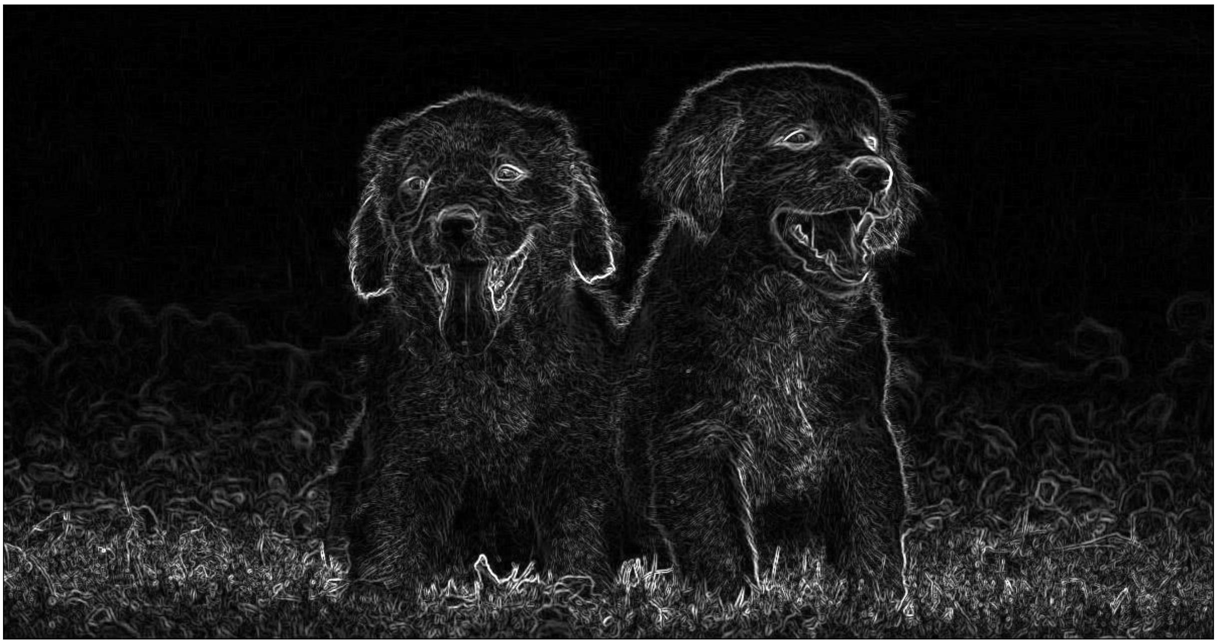
3. Fényerő növelése



16. ábra - Fényerő növelésének eredménye

A fényerő növelése aritmetikai szempontból egy egyszerű összeadás, azonban a szoftveres implementáció során kritikus volt a túlcsordulás (overflow) elkerülése. Ha egy már eleve világos (például 220-as értékű) pixelhez hozzáadunk 50-et, a hagyományos 8 bites tárolás esetén az érték átfordulna, és a fehérből irreális fekete képpont jönne létre. Ennek kiküszöbölésére a Rust saturating_add metódusát alkalmaztam, amely pontosan 255-nél maximalizálja az értéket. A kimeneti fotón látható, hogy a kép összességében világosabb lett, az eredetileg is világos területek pedig természetes módon elérték a maximális telítettséget anélkül, hogy grafikai hibák (artefaktok) keletkeztek volna.

4. Sobel élkeresés



17. ábra - Sobel-operátorral végzett élkeresés eredménye

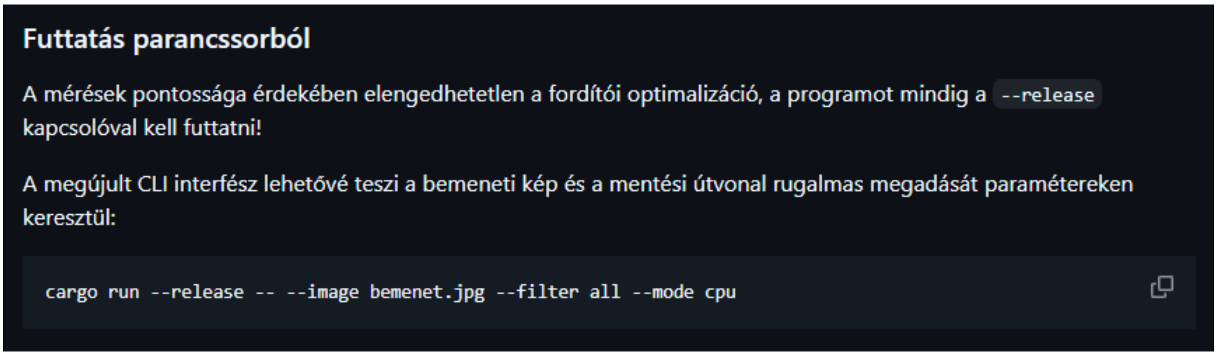
A Sobel élkereső a bemutatott szoftver egyik legkomplexebb algoritmusa, amely a számítógépes látásban (Computer Vision) is alapvető technológia. A művelet előtt a program a képet szürkeárnyalatossá alakítja, majd egy **3x3**-as konvolúciós mátrixszal (kernel) végigpásztázza a pixeleket. Az algoritmus vízszintes és függőleges irányban is vizsgálja a fényerő-gradienseket (a világos és sötét pixelek közötti hirtelen átmeneteket), majd ezek vektoros összegéből határozza meg az élek erősségét. A fekete hátterű kimeneti képen a kutyák körvonalai és a fűszálak kontúrjai élénk fehér színnel rajzolódnak ki, tökéletesen demonstrálva a magas frekvenciájú képi részletek detektálását.

5. Box Blur (Homályosítás)

18. ábra - Box Blur konvolúciós szűrő eredménye

A Box Blur algoritmus egy térbeli aluláteresztő szűrőként funkcionál. Működése során minden pixelt a saját és a közvetlen környezetében lévő 8 szomszédos pixel (tehát összesen egy **3x3**-as ablak) átlagértékével helyettesít. Mivel ez a folyamat csatornánként (RGB) külön-külön lefut, a művelet rendkívül számításigényes, de vizuálisan hatékonyan mossa el a részleteket. Az eredményül kapott képen a kutyák és a háttér éles kontúrjai elmosódtak, a formák lágyabbá váltak, ami az átlagoló konvolúció tipikus, elvárt viselkedése.

4.2. Futtatási környezet és parancssori paraméterezés



Futtatás parancssorból

A mérések pontossága érdekében elengedhetetlen a fordítói optimalizáció, a programot mindig a `--release` kapcsolóval kell futtatni!

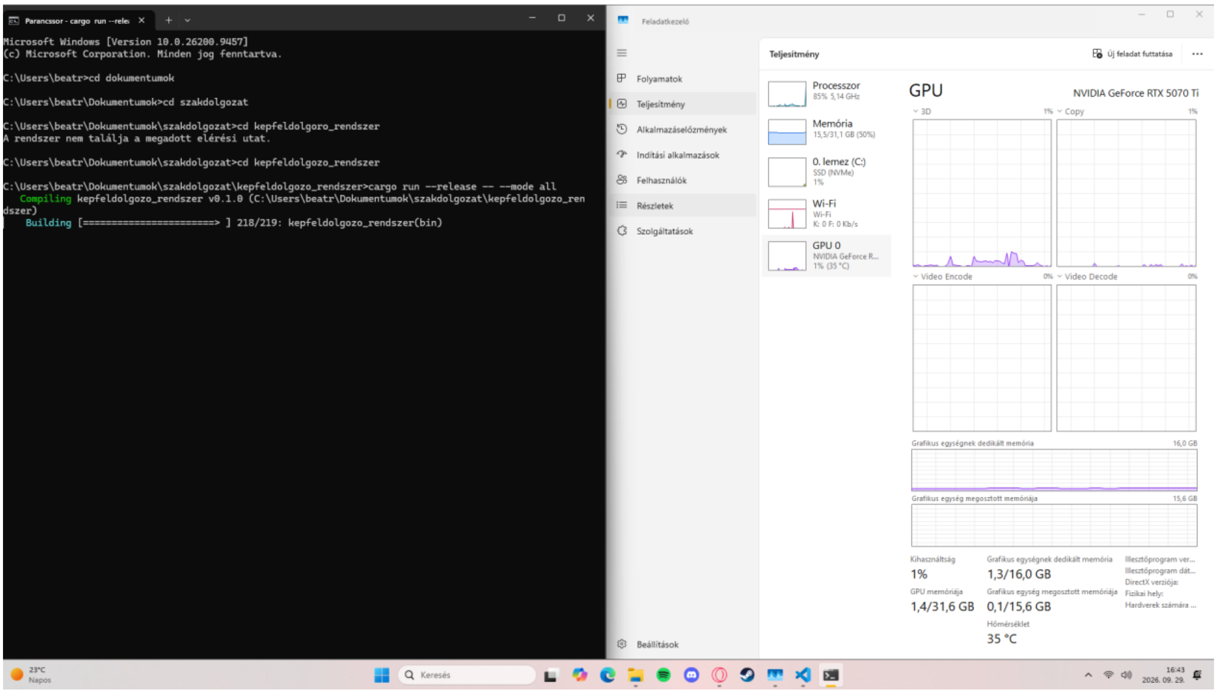
A megújult CLI interfész lehetővé teszi a bemeneti kép és a mentési útvonal rugalmas megadását paramétereken keresztül:

```
cargo run --release -- --image bemenet.jpg --filter all --mode cpu
```

19. ábra - A szoftver futtatási útmutatója és CLI paraméterei

A szoftver fejlesztése és tesztelése során kritikus szempont volt a mérések hitelességének és reprodukálhatóságának biztosítása. Ahogy a **README**-ben is rögzítettem, a Rust programok valós idejű teljesítménymérése kizárólag a fordító maximális optimalizációja mellett ad releváns képet. Ezt a Cargo csomagkezelő **--release** kapcsolójával értem el, amely eltávolítja a fejlesztői hibakereső (debug) szimbólumokat, és agresszív kódoptimalizálást hajt végre a bináris állományon.[19] A program indítása a modern szoftverfejlesztési elveknek megfelelően parancssorból (CLI) történik, ahol a felhasználó rugalmasan paraméterezheti a végrehajtást. A **--image** argumentum segítségével adható meg a bemeneti fájl elérési útja, a **--filter** paraméterrel választható ki a futtatandó matematikai transzformáció (a tesztek során ez jellemzően az **all** értéket kapta), míg a **--mode** flag explicit módon meghatározza a számítási architektúrát (például cpu vagy gpu). Ez a moduláris parancssori felépítés tette lehetővé a különböző hardveres megközelítések automatizált és pontos összehasonlítását.

4.3. Hardveres terhelésvizsgálat és a végrehajtási idő elemzése



```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. Minden jog fenntartva.

C:\Users\beatr>cd dokumentumok

C:\Users\beatr\Dokumentumok>cd szakdolgozat

C:\Users\beatr\Dokumentumok\szakdolgozat>cd kepfeldolgoro_rendszer
A rendszer nem találja a megadott elérési utat.

C:\Users\beatr\Dokumentumok\szakdolgozat>cd kepfeldolgozo_rendszer

C:\Users\beatr\Dokumentumok\szakdolgozat\kepfeldolgozo_rendszer>cargo run --release -- --mode all
   Compiling kepfeldolgozo_rendszer v0.1.0 (C:\Users\beatr\Dokumentumok\szakdolgozat\kepfeldolgozo_ren
dszer)
    Building [======================> ] 218/219: kepfeldolgozo_rendszer(bin)
```

Parancssor - cargo run --rele
Feladatkezelő
Folyamatok
Teljesítmény
Alkalmazáselőzmények
Indítási alkalmazások
Felhasználók
Részletek
Szolgáltatások
Beállítások
Teljesítmény
Új feladat futtatása
Processzor
85% 5,14 GHz
Memória
15,5/31,1 GB (50%)
0. lemez (C:)
SSD (NVMe)
1%
Wi-Fi
Wi-Fi
K: 0 F: 0 Kb/s
GPU 0
NVIDIA GeForce R...
1% (35 °C)
GPU
NVIDIA GeForce RTX 5070 Ti
3D
1%
Copy
1%
Video Encode
0%
Video Decode
0%
Grafikus egységnek dedikált memória
16,0 GB
Grafikus egység megosztott memóriája
15,6 GB
Kihasználtság
1%
Grafikus egységnek dedikált memória
1,3/16,0 GB
GPU memóriája
1,4/31,6 GB
Grafikus egység megosztott memóriája
0,1/15,6 GB
Hőmérséklet
35 °C
Illesztőprogram ver...
Illesztőprogram dát...
DirectX verziója:
Fizikai hely:
Hardverek számára ...
23°C
Napos
Keresés
16:43
2026. 09. 29.

20. ábra - A szoftver fordítási folyamata és a CPU maximális terheltsége

---

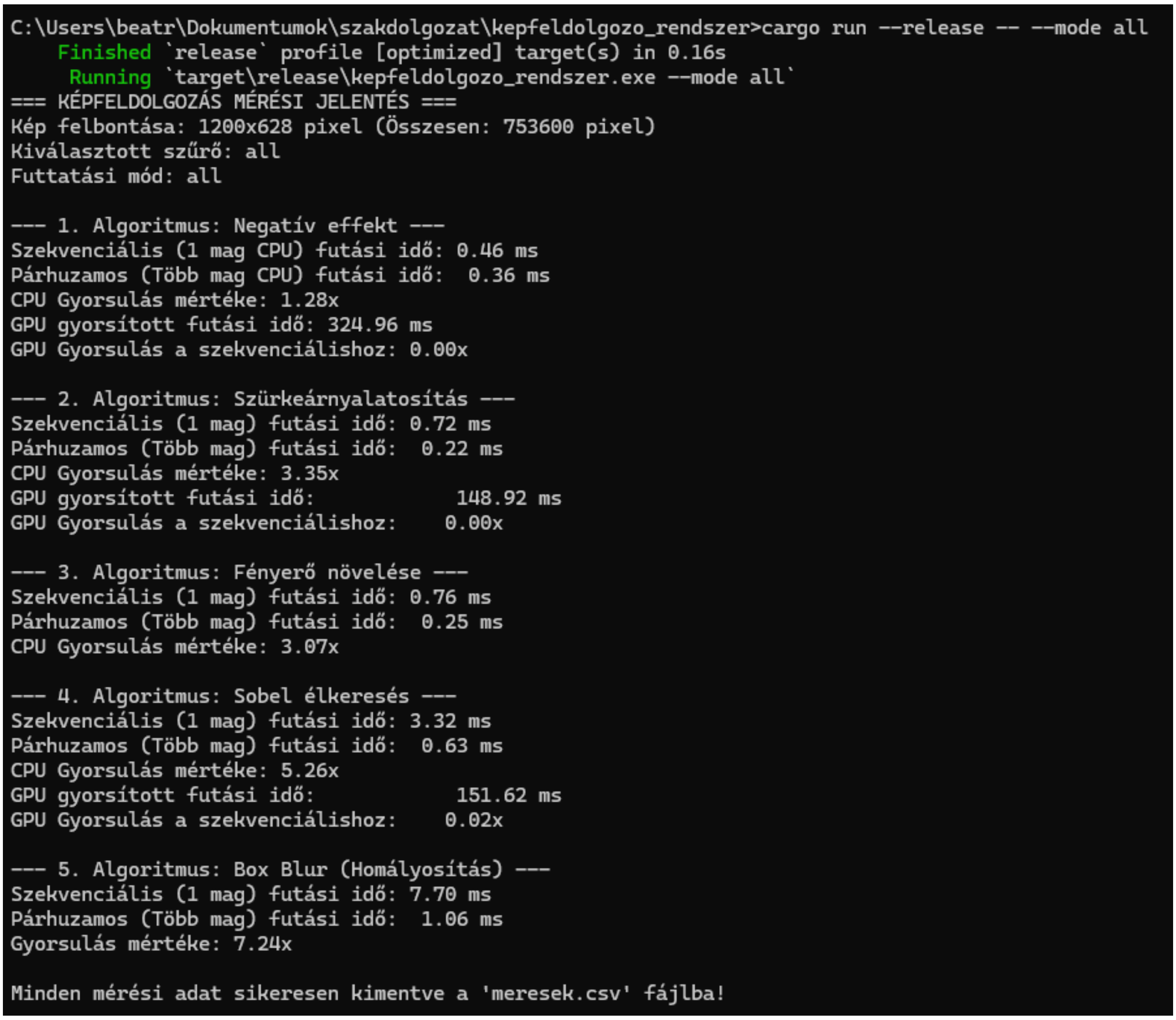
[19] The Cargo Book. Chapter 3.5: Profiles (Release). Rust Foundation. - https://doc.rust-lang.org/cargo/reference/profiles.html#release

A szoftver implementációjának tesztelése rávilágított a Rust nyelv, valamint a modern többmagos hardverek (AMD Ryzen 7 9800X3D) kombinált teljesítményére.

A **cargo run --release -- --mode all** parancs futtatását követően a rendszer először elvégzi a kód véglegesítésre szánt (release) optimalizált fordítását, majd végrehajtja a transzformációkat.

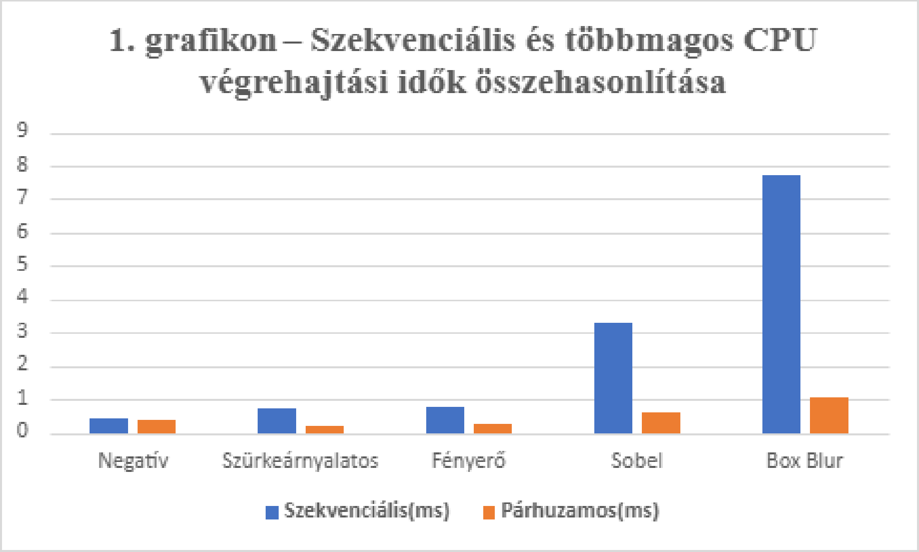
A fenti ábrán a Cargo csomagkezelő fordítási (Building) fázisa látható, amely agresszíven kihasználja a hardveres erőforrásokat a maximális kódoptimalizáció elérése érdekében. A folyamat során a Windows Feladatkezelő teljesítménygrafikonján egy erőteljes teljesítménycsúcs látható, ugyanis a processzor kihasználtsága 85%-ra ugrott, miközben az órajel elérte az extrém 5,14 GHz-es értéket. Ez a masszív terhelés tökéletesen igazolja a Rust fordítóprogramjának (rustc) modern, párhuzamosított felépítését. A memória (RAM) kihasználtsága a folyamat során optimális és stabil, 50%-os (15,5 GB) értéken maradt. A tényleges futtatás (amikor a fordítás befejeződik és a program feldolgozza a képeket) ehhez képest egy érdekes operációs rendszer szintű jelenséget mutatott. A matematikai transzformációk a hardveres gyorsításnak és a Rayon keretrendszernek köszönhetően milliszekundumos nagyságrendben futnak le. Mivel a Windows Feladatkezelője másodpercenként csupán egyszer mintavételez, a mindössze néhány milliszekundumig tartó futtatási terhelés vizuálisan már szinte nem is jelenik meg a grafikonon. Ez a mérési eredmény a gyakorlatban is alátámasztja a dolgozat korábbi elméleti megállapításait, mivel egy memóriabiztos, okosan többszálasított (work-stealing) kód a modern hardvereken gyakorlatilag várakozási idő nélkül képes nagyméretű adathalmazok párhuzamos feldolgozására.

4.4. Képfeldolgozási mérési jelentés és teljesítmény-elemzés



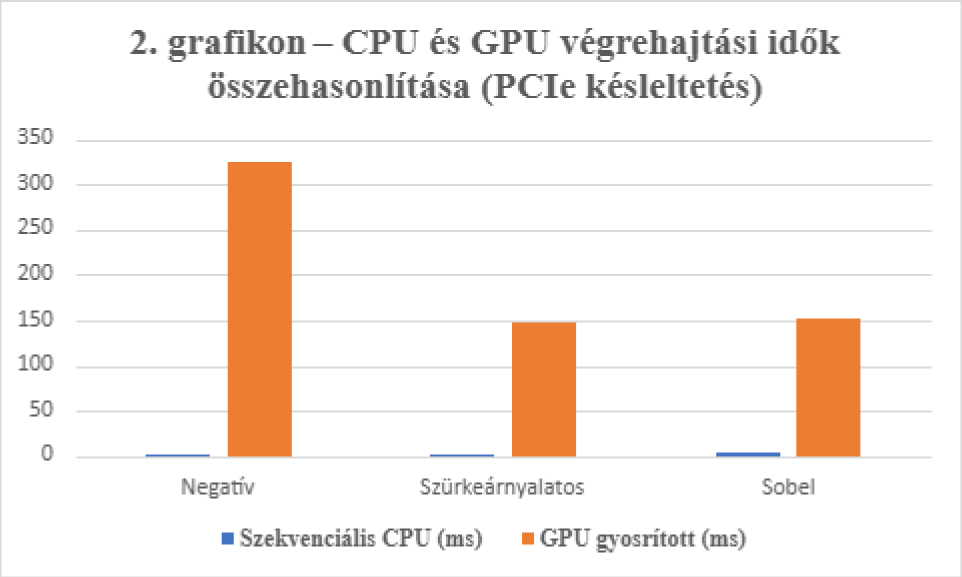
```
C:\Users\beatr\Dokumentumok\szakdolgozat\kepfeldolgozo_rendszer>cargo run --release -- --mode all
    Finished `release` profile [optimized] target(s) in 0.16s
     Running `target\release\kepfeldolgozo_rendszer.exe --mode all`
=== KÉPFELDOLGOZÁS MÉRÉSI JELENTÉS ===
Kép felbontása: 1200x628 pixel (Összesen: 753600 pixel)
Kiválasztott szűrő: all
Futtatási mód: all

--- 1. Algoritmus: Negatív effekt ---
Szekvenciális (1 mag CPU) futási idő: 0.46 ms
Párhuzamos (Több mag CPU) futási idő:  0.36 ms
CPU Gyorsulás mértéke: 1.28x
GPU gyorsított futási idő: 324.96 ms
GPU Gyorsulás a szekvenciálishoz: 0.00x

--- 2. Algoritmus: Szürkeárnyalatosítás ---
Szekvenciális (1 mag) futási idő: 0.72 ms
Párhuzamos (Több mag) futási idő:  0.22 ms
CPU Gyorsulás mértéke: 3.35x
GPU gyorsított futási idő:            148.92 ms
GPU Gyorsulás a szekvenciálishoz:    0.00x

--- 3. Algoritmus: Fényerő növelése ---
Szekvenciális (1 mag) futási idő: 0.76 ms
Párhuzamos (Több mag) futási idő:  0.25 ms
CPU Gyorsulás mértéke: 3.07x

--- 4. Algoritmus: Sobel élkeresés ---
Szekvenciális (1 mag) futási idő: 3.32 ms
Párhuzamos (Több mag) futási idő:  0.63 ms
CPU Gyorsulás mértéke: 5.26x
GPU gyorsított futási idő:            151.62 ms
GPU Gyorsulás a szekvenciálishoz:    0.02x

--- 5. Algoritmus: Box Blur (Homályosítás) ---
Szekvenciális (1 mag) futási idő: 7.70 ms
Párhuzamos (Több mag) futási idő:  1.06 ms
Gyorsulás mértéke: 7.24x

Minden mérési adat sikeresen kimentve a 'meresek.csv' fájlba!
```

21. ábra - A szoftver által generált konzolos mérési jelentés



22. grafikon - Szekvenciális és többmagos CPU végrehajtási idők összehasonlítása

A fenti oszlopdiagramon vizuálisan is egyértelműen megmutatkozik a párhuzamosítás hatékonysága a különböző matematikai komplexitású algoritmusok esetén. Jól látható, hogy az elemi, pontszerű pixelműveleteknél (Negatív effekt, Szürkeárnyalatosítás, Fényerő növelése) a szekvenciális végrehajtás (kék oszlop) is viszonylag gyors, 1 milliszekundum alatti futási időt eredményezett. Ezeknél az eljárásoknál a Rayon keretrendszerrel végzett többszálúsítás (narancssárga oszlop) bár hozott érdemi sebességnövekedést, a felgyorsulás relatív mértéke nem annyira drasztikus. Ennek informatikai oka, hogy az ilyen alacsony számításigényű műveleteknél a programszálak létrehozásának, menedzselésének és a memóriaszinkronizációnak az adminisztrációs ideje (overhead) már összemérhető magával a hasznos számítási idővel. Ezzel szemben a térbeli konvolúciót igénylő, magas számításigényű szűrőknél (Sobel élkeresés, Box Blur) a modern többmagos architektúra (AMD Ryzen 7 9800X3D) valódi ereje dominál. A Box Blur esetében a szekvenciális megközelítés 7,70 ms-os végrehajtási idejét a párhuzamosítás 1,06 ms-ra szorította le. A grafikonon látható hatalmas, több mint hétszeres időkülönbség vizuálisan is igazolja a dolgozat azon korábbi elméleti megállapítását, miszerint a komplex képfeldolgozási feladatoknál a feladatlopó (work-stealing) ütemezéssel ellátott többszálúsítás elengedhetetlen a skálázható és magas teljesítmény eléréséhez. A szoftveres implementáció lezárásaként a program egy részletes mérési jelentést generál a standard kimeneten (CLI), amely számszerűsíti az egyes algoritmusok végrehajtási idejét. A fenti ábrán egy 1200x628 pixel (összesen 753 600 képpont) felbontású bemeneti fájl teljes körű tesztelése látható. A rögzített milliszekundumos értékek drasztikus különbséget mutatnak a szekvenciális (egyszálas) és a párhuzamosított CPU futás között. A Rayon keretrendszerrel megvalósított többszálasítás hatékonysága a számításigényesebb algoritmusoknál mutatkozik meg a leginkább, a Sobel élkeresésnél 5,26-szoros, míg a komplex Box Blur (Homályosítás) eljárásnál lenyűgöző, 7,24-szeres gyorsulást sikerült elérni a szekvenciális végrehajtáshoz képest (7,70 ms-ról 1,06 ms-ra csökkentve a futásidőt). A mérési jegyzőkönyv rávilágít egy klasszikus hardveres architektúrális jelenségre is a GPU alapú gyorsítás kapcsán. Bár a videókártya számítási kapacitása elméletben jóval nagyobb, a viszonylag kis felbontású (1 megapixel alatti) képek esetében a GPU futási idők (pl. 324,96 ms a negatív effektnél) jelentősen elmaradnak a CPU teljesítményétől. Ennek az informatikai oka a PCI-Express sínen történő adatmozgatás (host-to-device és device-to-host memória másolás), valamint a WGPU inicializációs költsége (overhead). Kis adatmennyiségnél ez a kommunikációs többletidő dominál, így a GPU-s gyorsítás valódi előnye majd csak a több tíz megapixeles (4K/8K) képek vagy folyamatos videóstreamek valós idejű feldolgozásánál térülne meg. A szoftver a futás legvégén a mért adatokat sikeresen exportálja egy meresek.csv állományba, ami lehetővé teszi a későbbiekben a teljesítményadatok külső szoftverekkel (pl. Excel) történő aggregálását és grafikus vizualizációját.



23. grafikon - CPU és GPU végrehajtási idők összehasonlítása (PCIe késleltetés)

4.5. A szoftver korlátai és jövőbeli továbbfejlesztési lehetőségek

Bár a kifejlesztett képfeldolgozó alkalmazás sikeresen bizonyította a Rust nyelv, valamint a többmagos (CPU) és a videókártyás (GPU) architektúrák közötti teljesítménybeli különbségeket, jelenlegi formájában egy kutatási célú prototípusnak tekinthető. A szoftver felépítése és technológiai alapjai rendkívül stabilak, azonban ipari vagy széleskörű kereskedelmi felhasználásához érdemes számba venni a jelenlegi korlátokat és az ezekből fakadó továbbfejlesztési lehetőségeket.

Felhasználói felület (GUI) hiánya

A jelenlegi implementáció egy parancssori (CLI) alkalmazás. Bár a teljesítménymérésekhez, a precíz paraméterezéshez és az automatizált kötegelt feldolgozáshoz (batch processing) ez a legoptimálisabb és leggyorsabb választás, a végfelhasználói élmény szempontjából korlátozott. Egy logikus jövőbeli fejlesztési irány egy asztali grafikus felület integrálása. A Rust ökoszisztémában egyre érettebb natív GUI keretrendszerek (például az egui vagy a Tauri) beépítésével a felhasználók vizuálisan, csúszkák segítségével állíthatnák be a Sobel-operátor érzékenységét vagy a Box Blur intenzitását, és valós időben láthatnák a transzformációk eredményét a képernyőn.

Valós idejű videófeldolgozás és a PCIe szűk keresztmetszet

Ahogy az a mérési jegyzőkönyv kiértékelésénél (4.4. fejezet) egyértelműen kiderült, a GPU-s gyorsítás legnagyobb hátránya a szimpla, statikus képek feldolgozásakor a PCI-Express sínen történő adatmozgatás (a RAM és a VRAM közötti oda-vissza másolás) adminisztrációs ideje. Egy jövőbeli fejlesztés során a szoftvert át lehetne alakítani valós idejű videóstreamek (például egy csatlakoztatott webkamera képének) feldolgozására. Ebben a forgatókönyvben a videókártya textúra-memóriája folyamatosan nyitva maradna, a számítási csővezetéket (compute pipeline) nem kellene minden képkockánál újra inicializálni. Így a GPU brutális párhuzamosítási képessége zökkenőmentesen biztosítana egy stabil, 60 FPS (képkocka/másodperc) sebességű élkeresést, amit a processzor önmagában már nem tudna akadásmentesen kiszolgálni.

Memóriakorlátok és ipari méretű adathalmazok

Jelenleg a program a beolvasott bemeneti képet egyetlen hatalmas, egydimenziós tömbként tölti be a központi memóriába. Ez a megközelítés a standard 4K vagy 8K felbontású fotókig tökéletesen és villámgyorsan működik. Ugyanakkor speciális, ipari környezetben – például több tíz gigabájtos orvosi MRI felvételek, vagy rendkívül nagy felbontású műholdképek esetén – ez a módszer memóriatúlcsorduláshoz (Out-Of-Memory, OOM) vezethet. A szoftver architektúrájának robusztusabbá tétele érdekében bevezethető lenne egy "darabolásos" (chunk-based vagy tile-based) adatfeldolgozási modell. Ilyenkor a rendszer csak a kép egy-egy kisebb szegmensét (csempéjét) töltené be a VRAM-ba, ott elvégezné a konvolúciót, visszamentené a háttértárra, majd lapozna a következőre. Ezáltal a szoftver elméletben végtelen felbontású képeket is képes lenne feldolgozni a hardver fizikai memóriakapacitásától teljesen függetlenül.

5. Összefoglalás

A szakdolgozatom célja egy párhuzamosított képfeldolgozó rendszer fejlesztése és teljesítményvizsgálata volt Rust programozási nyelven. A feladat során nem csupán egy funkcionális parancssori alkalmazást hoztam létre, hanem részletesen vizsgáltam a modern többmagos processzorok (CPU) és a videókártyák (GPU) architekturális különbségeiből származó teljesítménybeli eltéréseket. A kutatómunka és a fejlesztés során egyértelművé vált, hogy a hagyományos, egyszálas szoftverek már nem képesek hatékonyan kihasználni a mai hardverek képességeit. A Rust nyelv memóriabiztos és adatütközéseket fordítási időben kizáró (ownership) modellje kiváló alapot biztosított a Rayon könyvtárral végzett CPU alapú többszálúsításhoz. A mérési eredmények egyértelműen megmutatták, hogy a feladatlopó ütemezés használatával az elemi pixelműveletek és a komplex konvolúciós algoritmusok futási ideje a processzormagok számával arányosan, drasztikusan csökkenthető. A legjelentősebb technológiai kihívást a wgpu keretrendszerre épülő, WGSL nyelven megírt Compute Shaderek implementálása jelentette. Bár a rendszermemória (RAM) és a videókártya (VRAM) közötti adatmásolás kisebb felbontásoknál adminisztrációs többletidőt (overhead) jelent, a térbeli konvolúciós szűrőknél (mint a Sobel élkereső vagy a Box Blur homályosítás) a GPU tömegesen párhuzamosított feldolgozása egyértelmű előnyt mutatott. Az alkalmazott kétdimenziós szálkiosztás és a bit-szintű memóriakezelés igazolta a hardverközeli optimalizáció létjogosultságát. Összességében a kitűzött célokat sikeresen megvalósítottam. A kifejlesztett alkalmazás stabilan és biztonságosan működik, a dokumentált mérési eredmények pedig gyakorlati bizonyítékot szolgáltatnak a megfelelő hardveres gyorsítási architektúra kiválasztásához. Bár a szoftver rendelkezik korlátokkal, a lefektetett technológiai alapok és a jövőbeli továbbfejlesztési irányok, mint például a nagyméretű orvosi adathalmazok vagy a valós idejű videóstreamek feldolgozása, további komoly kutatási potenciált rejtenek.

6. Irodalomjegyzék

1. Bohr, Mark (2007): A 30 Year Retrospective on Dennard's Scaling Theory. IEEE Solid-State Circuits Society Newsletter. Elérhető: https://www.eng.auburn.edu/~agrawvd/COURSE/READING/LOWP/Boh07.pdf, Letöltés dátuma: 2026. 02. 15.

2. Bytemuck hivatalos dokumentáció. Elérhető: https://docs.rs/bytemuck/, Letöltés dátuma: 2026. 05. 10.

3. Gonzalez, Rafael C. – Woods, Richard E. (2018): Digital Image Processing (4th Edition). Pearson. Elérhető: https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf, Letöltés dátuma: 2026. 02. 20.

4. ITU-R Recommendation BT.601-7 (2011): Studio encoding parameters of digital television. International Telecommunication Union. Elérhető: https://www.itu.int/rec/R-REC-BT.601/en, Letöltés dátuma: 2026. 04. 05.

5. Kirk, David B. – Hwu, Wen-mei W. (2022): Programming Massively Parallel Processors: A Hands-on Approach (4th Edition). Morgan Kaufmann.

6. Klabnik, Steve – Nichols, Carol (2023): The Rust Programming Language. No Starch Press. Elérhető: https://doc.rust-lang.org/book/, Letöltés dátuma: 2026. 02. 12.

7. Klabnik, Steve – Nichols, Carol (2023): The Rust Programming Language. Chapter 9: Error Handling. No Starch Press. Elérhető: https://doc.rust-lang.org/book/ch09-00-error-handling.html, Letöltés dátuma: 2026. 05. 15.

8. Patterson, David A. – Hennessy, John L. (2013): Computer Organization and Design: The Hardware/Software Interface (5. kiadás). Morgan Kaufmann.

9. Rayon hivatalos dokumentáció. Elérhető: https://docs.rs/rayon/latest/rayon/, Letöltés dátuma: 2026. 04. 18.

10. Rust Foundation (é. n.): The Cargo Book. Chapter 3.5: Profiles (Release). Elérhető: https://doc.rust-lang.org/cargo/reference/profiles.html#release, Letöltés dátuma: 2026. 08. 14.

11. Sutter, Herb (2005): The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. Dr. Dobb's Journal. Elérhető: http://www.gotw.ca/publications/concurrency-ddj.htm, Letöltés dátuma: 2026. 02. 28.

12. Vangala, Rohith Reddy (2025): Decoding CPU vs. GPU: A Detailed Exploration of NVIDIA and AMD GPU Architectures. Medium. Elérhető: https://medium.com/@rohithreddy66666/decoding-cpu-vs-gpu-a-detailed-exploration-of-nvidia-and-amd-gpu-architectures-e09ebfb594ed, Letöltés dátuma: 2026. 03. 10.

13. W3C WebGPU Working Group (2024): WebGPU Shading Language (WGSL) Specification. W3C. Elérhető: https://www.w3.org/TR/WGSL/, Letöltés dátuma: 2026. 05. 22.

14. WGPU hivatalos weboldal és fejlesztői leírás. Elérhető: https://wgpu.rs és https://docs.rs/wgpu, Letöltés dátuma: 2026. 05. 25.

15. Williams, Anthony (2019): C++ Concurrency in Action (Second Edition). Manning Publications.

7. Ábrajegyzék

1. ábra - A CPU és a GPU belső architektúrájának strukturális összehasonlítása 9
2. ábra - A parancssori argumentumok (CLI) definíciója a clap csomag segítségével 13
3. ábra - A teszteléshez használt 1200x628 pixeles, 24 bites RGB bemeneti kép 13
4. ábra - A szekvenciális iteráció és a Rayon alapú párhuzamosított adatfeldolgozás szintaktikai összehasonlítása 15
5. ábra - A VRAM tároló puffer (storage buffer) és a CPU által olvasható átmeneti puffer (staging buffer) inicializálása 16
6. ábra - A negatív transzformációt végrehajtó, tömegesen párhuzamosított WGSL Compute Shader kódja 17
7. ábra - Az aszinkron memórialeképezés (map_async) folyamata a VRAM és a rendszermemória között 17
8. ábra - A Sobel-operátor kétdimenziós szálkiosztása 18
9. ábra - A 32 bites memóriablokkból 8 bites képpontokat kicsomagoló WGSL segédfüggvény 19
10. ábra - A kétdimenziós konvolúciós mátrix (Sobel-operátor) futtatása a GPU magokon ... 19
11. ábra - Fájlbeolvasás biztonságos hibakezelése mintaillesztéssel (match) 20
12. ábra - A videókártyás futtatás eredményének biztonságos lekezelése és az aszinkron végrehajtás szinkronizálása 21
13. ábra - Algoritmus, Szekvenciális (ms), Párhuzamos / GPU (ms), Gyorsulás) 22
14. ábra - Negatív transzformáció eredménye 23
15. ábra - Szürkeárnyalatos (Luma) konverzió eredménye 24
16. ábra - Fényerő növelésének eredménye 25
17. ábra - Sobel-operátorral végzett élkeresés eredménye 26
18. ábra - Box Blur konvolúciós szűrő eredménye 27
19. ábra - A szoftver futtatási útmutatója és CLI paraméterei 27
20. ábra - A szoftver fordítási folyamata és a CPU maximális terheltség 28
21. ábra - A szoftver által generált konzolos mérési jelentés 30
22. grafikon - Szekvenciális és többmagos CPU végrehajtási idők összehasonlítása 31
23. grafikon - CPU és GPU végrehajtási idők összehasonlítása (PCIe késleltetés) 32